

SISTEMA DE GESTIÓN INTEGRADO

SGI - Metalúrgica

Escuela Técnica 32 "General José de San Martín"



INFORMACIÓN DE ALUMNO - PROYECTO

ESCUELA	Escuela Técnica nº32 Gral. José de San Martín
ALUMNO	Ramiro Garcia Guchea
CURSO	6to 1da Computacion
DURACIÓN	37 días (20 mayo - 26 junio)
ESTADO	Completado
LÍNEAS CÓDIGO	~15,000+
MÓDULOS	8 funcionales
Fecha de Creación	20 de mayo de 2026

- **Resumen Ejecutivo**

- El presente proyecto consiste en el desarrollo de un sistema web de gestión para una empresa metalúrgica, orientado a centralizar y optimizar los procesos administrativos y operativos vinculados con usuarios, clientes, productos, stock, pedidos y producción. La solución fue diseñada para reemplazar tareas manuales y dispersas por una plataforma digital que permita mejorar el control de la información, reducir errores y agilizar la toma de decisiones.

El sistema fue implementado utilizando React en el frontend, FastAPI en el backend y PostgreSQL como base de datos, integrando además autenticación segura mediante JWT y servicios de Supabase para la gestión de datos y despliegue. La aplicación cuenta con módulos diferenciados para la administración de usuarios, clientes, productos, stock, pedidos y reportes, con una interfaz intuitiva y adaptable a las necesidades del entorno de trabajo.

Como resultado, se obtuvo una herramienta funcional, escalable y segura, capaz de mejorar la organización interna de la empresa y facilitar el seguimiento de sus operaciones. Este proyecto no solo resuelve una necesidad concreta del

negocio, sino que también demuestra la aplicación práctica de tecnologías modernas en un contexto real de gestión industrial.

● ÍNDICE GENERAL

● 1. Introducción

- 1.1 Presentación del Proyecto
- 1.2 Contexto de la Empresa Metalúrgica
- 1.3 Problemática Detectada
- 1.4 Objetivos Generales
- 1.5 Objetivos Específicos
- 1.6 Alcance del Proyecto

● 2. Relevamiento y Análisis de Requerimientos

- 2.1 Análisis de la Problemática
- 2.2 Requerimientos Funcionales
- 2.3 Requerimientos No Funcionales
- 2.4 Casos de Uso Principales
- 2.5 Actores del Sistema
- 2.6 Procesos del Negocio

● 3. Planificación del Proyecto

- 3.1 Metodología de Trabajo
- 3.2 Organización del Equipo
- 3.3 Cronograma de Actividades
- 3.4 Diagrama de Gantt
- 3.5 Gestión de Riesgos
- 3.6 Seguimiento y Control del Proyecto

● 4. Arquitectura General del Sistema

- 4.1 Arquitectura Cliente-Servidor
- 4.2 Arquitectura Frontend
- 4.3 Arquitectura Backend
- 4.4 Arquitectura de Base de Datos

4.5 Flujo General de Información

4.6 Integración de Componentes

- **5. Tecnologías Utilizadas**

5.1 React

5.2 FastAPI

5.3 PostgreSQL

5.4 Supabase

5.5 JWT

5.6 Node.js

5.7 Librerías y Dependencias Complementarias

- **6. Diseño de la Base de Datos**

6.1 Modelo Entidad-Relación

6.2 Tabla Usuarios

6.3 Tabla Roles

6.4 Tabla Clientes

6.5 Tabla Productos

6.6 Tabla Componentes

6.7 Tabla Stock

6.8 Tabla Pedidos

6.9 Tabla Detalle de Pedidos

6.10 Tabla Movimientos de Stock

6.11 Relaciones e Integridad Referencial

- **7. Desarrollo Backend**

7.1 Introducción al Backend

7.2 Estructura del Proyecto

7.3 Configuración Inicial

7.4 Gestión de Autenticación

7.5 Gestión de Usuarios

7.6 Gestión de Clientes

7.7 Gestión de Productos

7.8 Gestión de Stock

7.9 Gestión de Pedidos

7.10 Gestión de Producción

7.11 Gestión de Reportes

7.12 Manejo de Excepciones

7.13 Validaciones y Seguridad
7.14 API REST Implementada

- **8. Desarrollo Frontend**

- 8.1 Introducción al Frontend
- 8.2 Arquitectura de Componentes
- 8.3 Sistema de Navegación
- 8.4 Gestión de Estados
- 8.5 Pantalla de Login
- 8.6 Dashboard Principal
- 8.7 Módulo de Usuarios
- 8.8 Módulo de Clientes
- 8.9 Módulo de Productos
- 8.10 Módulo de Stock
- 8.11 Módulo de Pedidos
- 8.12 Módulo de Producción
- 8.13 Módulo de Reportes
- 8.14 Componentes Reutilizables
- 8.15 Hooks Personalizados

- **9. Seguridad del Sistema**

- 9.1 Autenticación JWT
- 9.2 Autorización por Roles
- 9.3 Protección de Rutas
- 9.4 Validación de Datos
- 9.5 Seguridad de la Información

- **10. Testing y Control de Calidad**

- 10.1 Estrategia de Testing
- 10.2 Pruebas de Login y Acceso de Usuarios
- 10.3 Pruebas Funcionales
- 10.4 Pruebas de Integración
- 10.5 Resultados Obtenidos
- 10.6 Corrección de Errores Detectados

- **11. Manual de Usuario**

- 11.1 Acceso al Sistema
- 11.2 Gestión de Usuarios
- 11.3 Gestión de Clientes
- 11.4 Gestión de Productos
- 11.5 Gestión de Stock
- 11.6 Gestión de Pedidos
- 11.7 Seguimiento de Producción
- 11.8 Generación de Reportes
- 11.9 Diagrama de Caso de uso

- **12. Manual de Deploy**

- 12.1 Requisitos Previos
- 12.2 Configuración del Backend
- 12.3 Configuración del Frontend
- 12.4 Configuración de la Base de Datos
- 12.5 Variables de Entorno
- 12.6 Publicación en Producción

- **13. Resultados del Proyecto**

- 13.1 Funcionalidades Implementadas
- 13.2 Comparación con los Requerimientos Iniciales
- 13.3 Beneficios Obtenidos

- **14. Conclusiones**

- 14.1 Objetivos Alcanzados
- 14.2 Aprendizajes Adquiridos
- 14.3 Dificultades Encontradas
- 14.4 Mejoras Futuras

- **15. Bibliografía**

- 15.1 Documentación Oficial
- 15.2 Recursos de Aprendizaje
- 15.3 Herramientas Utilizadas
- 15.4 Referencias Académicas

- **16 Código**

- 16.1 Repositorio de GitHub

- 16.2 Diagrama de Secuencia.

1. INTRODUCCIÓN

1.1 Presentación del Proyecto

El presente trabajo documenta el desarrollo de un Sistema de Gestión Integrado (SGI) diseñado para una empresa del sector metalúrgico. El proyecto fue realizado en el marco de las Prácticas Profesionalizantes de la Escuela Técnica N.º 32 “Gral. José de San Martín”, con el objetivo de aplicar conocimientos adquiridos durante la formación técnica en un caso de uso real y representativo del ámbito empresarial.

La solución desarrollada consiste en una aplicación web que centraliza los procesos administrativos y operativos de la organización, permitiendo gestionar usuarios, clientes, productos, stock, pedidos, producción y reportes desde una única plataforma. El sistema fue concebido para brindar información en tiempo real, mejorar el control de los recursos y optimizar la toma de decisiones dentro de la empresa.

Para su implementación se utilizaron tecnologías modernas ampliamente utilizadas en el desarrollo de software profesional. El frontend fue desarrollado con React, el backend con FastAPI y la base de datos con PostgreSQL mediante la plataforma Supabase. Además, se implementó un sistema de autenticación basado en JSON Web Tokens (JWT) para garantizar la seguridad y el control de acceso a la información.

La finalidad principal del proyecto fue construir una herramienta funcional, escalable y segura que permitiera resolver problemas operativos reales y demostrar la aplicación práctica de metodologías, herramientas y tecnologías estudiadas durante la carrera técnica.

1.2 Contexto de la Empresa Metalúrgica

Las empresas metalúrgicas se caracterizan por trabajar con una amplia variedad de materias primas, componentes y productos terminados destinados a diferentes clientes y proyectos. Debido a la naturaleza de su actividad, resulta fundamental

contar con mecanismos de control que permitan administrar eficientemente el stock disponible, realizar un seguimiento de los pedidos y coordinar adecuadamente los procesos productivos.

En este contexto, la gestión de la información adquiere un papel fundamental. La falta de centralización de los datos puede generar errores en el control de inventario, retrasos en la producción, pérdidas económicas y dificultades para responder a las necesidades de los clientes.

La empresa considerada para este proyecto requiere gestionar productos simples y productos compuestos, es decir, artículos que se encuentran conformados por varios componentes o materias primas. Esta característica introduce una complejidad adicional, ya que la disponibilidad de un producto final depende directamente de la disponibilidad de cada uno de sus componentes.

Asimismo, la organización necesita disponer de información actualizada sobre el estado de los pedidos, el avance de producción, el movimiento de stock y las operaciones realizadas por los distintos usuarios del sistema.

Por este motivo, surge la necesidad de desarrollar una solución informática que permita integrar todos estos procesos dentro de una única plataforma accesible desde cualquier navegador web.

1.3 Problemática Detectada

Durante el relevamiento inicial se identificaron diversas dificultades asociadas a la gestión operativa de la empresa.

Uno de los principales problemas detectados fue la falta de visibilidad en tiempo real sobre el stock disponible. La información se encontraba dispersa o dependía de registros manuales, dificultando la toma de decisiones y aumentando el riesgo de errores en los pedidos.

Otro inconveniente importante estaba relacionado con la gestión de productos compuestos. Determinar cuántas unidades de un producto podían fabricarse requería analizar manualmente la disponibilidad de cada componente, generando demoras y aumentando la posibilidad de inconsistencias.

También se observaron dificultades en el seguimiento de pedidos. La empresa necesitaba conocer en todo momento el estado de cada solicitud, desde su creación hasta la entrega final, incluyendo las etapas de fabricación y control de calidad.

Además, no existía un sistema centralizado para administrar usuarios, roles y permisos, lo que limitaba el control sobre las acciones realizadas por cada integrante de la organización.

Por último, se identificó la ausencia de reportes e indicadores que permitieran analizar la situación operativa de la empresa, evaluar el rendimiento de los procesos y facilitar la toma de decisiones estratégicas.

Estas problemáticas evidenciaron la necesidad de implementar una solución tecnológica capaz de integrar la información y automatizar los procesos clave del negocio.

1.4 Objetivo General

Desarrollar un Sistema de Gestión Integrado (SGI) para una empresa metalúrgica que permita centralizar la administración de usuarios, clientes, productos, stock, pedidos y producción, proporcionando información en tiempo real y optimizando los procesos operativos de la organización.

1.5 Objetivos Específicos

- Diseñar una plataforma web accesible desde cualquier navegador moderno.
- Implementar un sistema de autenticación y autorización basado en roles y permisos.
- Desarrollar herramientas para la gestión integral de usuarios y clientes.
- Administrar productos simples y productos compuestos dentro de un catálogo centralizado.
- Controlar el stock disponible mediante registros de movimientos y alertas automáticas.
- Implementar mecanismos de validación de stock durante la generación de pedidos.
- Automatizar la actualización de inventario según los cambios de estado de los pedidos.
- Facilitar el seguimiento visual del proceso de producción mediante un monitor especializado.
- Generar reportes e indicadores para el análisis de la información operativa.

- Garantizar la integridad y seguridad de los datos almacenados.
 - Aplicar buenas prácticas de desarrollo de software utilizando tecnologías modernas.
-

1.6 Alcance del Proyecto

El alcance del proyecto comprende el análisis, diseño, desarrollo, integración y documentación de un sistema web completo destinado a la gestión de una empresa metalúrgica.

La solución implementada permite administrar usuarios, clientes, productos, stock, pedidos y procesos productivos desde una única plataforma centralizada. Asimismo, incorpora mecanismos de autenticación, control de acceso, generación de reportes y visualización de información en tiempo real.

El sistema contempla la gestión de productos simples y compuestos, la validación automática de disponibilidad de stock, el seguimiento de pedidos mediante distintos estados operativos y la generación de información útil para la toma de decisiones.

No forman parte del alcance del proyecto la integración con sistemas contables externos, pasarelas de pago, sistemas bancarios o aplicaciones de terceros. Estas funcionalidades podrán ser consideradas como futuras mejoras y ampliaciones del sistema.

El proyecto fue desarrollado siguiendo una arquitectura cliente-servidor moderna y escalable, permitiendo futuras adaptaciones y la incorporación de nuevas funcionalidades sin necesidad de rediseñar completamente la solución.

2. RELEVAMIENTO Y ANÁLISIS DE REQUERIMIENTOS

2.1 Análisis de la Problemática

Antes de iniciar el desarrollo del sistema fue necesario realizar un proceso de aprendizaje y adaptación a nuevas tecnologías. Uno de los principales desafíos que

enfrenté durante esta etapa fue estudiar lenguajes de programación que no conocía previamente, ya que no contaba con conocimientos sólidos en varios de ellos.

Esta situación generó diversas dificultades al momento de armar el sitio web, debido a que tuve que comprender desde cero la lógica de funcionamiento, la sintaxis y la forma en que se integraban las distintas herramientas utilizadas en el proyecto. En varias ocasiones esto provocó demoras, errores en la implementación y la necesidad de revisar constantemente documentación y ejemplos para poder avanzar correctamente.

A pesar de estos inconvenientes, el proceso resultó muy valioso porque me permitió adquirir nuevos conocimientos y fortalecer mis habilidades técnicas. Con el tiempo, fui logrando mayor comprensión sobre los lenguajes utilizados y pude resolver los problemas que se presentaban durante el desarrollo.

A partir de esta experiencia se determinó la importancia de organizar mejor el trabajo, investigar cada tecnología antes de implementarla y avanzar de manera progresiva para poder construir el sitio web de forma correcta y funcional.

2.2 Requerimientos Funcionales

Los requerimientos funcionales describen las acciones y servicios que el sistema debe proporcionar para satisfacer las necesidades de los usuarios y de la organización.

Área funcional	Requerimientos
Gestión de Usuarios	El sistema debe permitir la creación, modificación y administración de usuarios por parte de los administradores del sistema.
Gestión de Usuarios	Cada usuario debe poseer un rol específico que determine las funcionalidades a las que puede acceder. Entre los roles definidos se encuentran administrador, vendedor, almacén, producción y consulta.
Gestión de Usuarios	Asimismo, el sistema debe permitir habilitar o deshabilitar usuarios sin necesidad de eliminarlos definitivamente, conservando la integridad histórica de la información.
Gestión de Clientes	El sistema debe permitir registrar clientes nuevos, consultar sus datos y realizar modificaciones cuando sea necesario.
Gestión de Clientes	La información almacenada debe facilitar la gestión de pedidos y el acceso al historial comercial de cada cliente.
Gestión de Productos	El sistema debe administrar tanto productos simples como productos compuestos.
Gestión de Productos	Los productos simples representan materias primas o componentes individuales, mientras que los productos compuestos están formados por uno o más productos simples.
Gestión de Productos	Cada producto debe poseer un código identificador, descripción, unidad de medida, stock disponible y stock mínimo.
Gestión de Stock	El sistema debe controlar el ingreso, egreso y ajuste de stock de cada producto.
Gestión de Stock	Cada movimiento realizado debe quedar registrado en un historial para garantizar la trazabilidad de las operaciones.
Gestión de Stock	Además, el sistema debe generar alertas automáticas cuando el stock disponible alcance niveles críticos o se encuentre por debajo del mínimo establecido.
Gestión de Listas de Precio	La aplicación debe permitir administrar múltiples listas de precios para diferentes tipos de clientes.

Gestión de Listas de Precio	Estas listas podrán ser asignadas a clientes específicos o utilizadas durante la generación de pedidos.
Gestión de Pedidos	El sistema debe permitir registrar pedidos de venta asociados a clientes y listas de precios.
Gestión de Pedidos	Durante la carga de un pedido se deberá validar automáticamente la disponibilidad de stock de los productos seleccionados.
Gestión de Pedidos	También deberá contemplarse la modificación de pedidos y el seguimiento de su evolución a través de distintos estados operativos.
Seguimiento de Producción	La solución debe incorporar una vista especializada para monitorear el estado de los pedidos activos.
Seguimiento de Producción	Esta funcionalidad permitirá visualizar en tiempo real el avance de la producción y facilitar la coordinación de las tareas operativas.
Reportes e Informes	El sistema deberá generar distintos tipos de reportes relacionados con clientes, pedidos y stock.
Reportes e Informes	Los informes deberán poder visualizarse en pantalla y exportarse para su posterior utilización administrativa.

2.3 Requerimientos No Funcionales

Los requerimientos no funcionales establecen características de calidad que el sistema debe cumplir independientemente de las funcionalidades implementadas.

Requerimiento	Descripción
Usabilidad	La interfaz debe ser intuitiva y fácil de utilizar para usuarios con diferentes niveles de experiencia informática.
Rendimiento	El sistema debe responder de forma rápida ante las operaciones habituales, minimizando tiempos de espera y mejorando la experiencia de uso.
Escalabilidad	La arquitectura implementada debe permitir la incorporación de nuevas funcionalidades sin afectar el funcionamiento existente.

Seguridad	Toda la información deberá protegerse mediante mecanismos de autenticación y autorización. El acceso a los recursos del sistema deberá estar restringido según el rol asignado a cada usuario.
Disponibilidad	La aplicación deberá encontrarse disponible para los usuarios autorizados siempre que exista conectividad a Internet.
Mantenibilidad	El código fuente deberá estar organizado siguiendo buenas prácticas de desarrollo para facilitar futuras tareas de mantenimiento y evolución.

2.4 Casos de Uso Principales

Los casos de uso representan las interacciones entre los usuarios y el sistema.

Los principales casos de uso identificados fueron:

- Iniciar sesión.
 - Gestionar usuarios.
 - Gestionar clientes.
 - Gestionar productos.
 - Gestionar stock.
 - Registrar movimientos de inventario.
 - Crear pedidos.
 - Modificar pedidos.
 - Actualizar estados de producción.
 - Consulta de reportes.
 - Exportar información.
 - Administrar listas de precios.
-

2.5 Actores del Sistema

Se identificaron diferentes tipos de usuarios con responsabilidades específicas dentro del sistema.

Actor	Descripción
Administrador	Responsable de gestionar usuarios, permisos y configuraciones generales.
Vendedor	Encargado de registrar clientes, generar pedidos y realizar consultas relacionadas con ventas.
Personal de Almacén	Responsable del control de inventario y de los movimientos de stock.
Personal de Producción	Encargado del seguimiento y actualización del estado de fabricación de los pedidos.
Gerencia	Usuario con acceso a consultas, reportes e indicadores estratégicos.

2.6 Procesos del Negocio

El sistema fue diseñado para acompañar el flujo operativo habitual de la empresa.

El proceso comienza con el registro de clientes y la administración del catálogo de productos disponibles.

Posteriormente se generan pedidos asociados a clientes específicos. Durante esta etapa se valida automáticamente la disponibilidad de stock.

Una vez aprobado el pedido, el sistema actualiza los estados correspondientes y registra los movimientos necesarios en el inventario.

Durante la producción, los responsables pueden consultar el monitor de seguimiento para visualizar el avance de cada pedido.

Finalmente, los usuarios autorizados pueden acceder a reportes e indicadores que facilitan el control operativo y la toma de decisiones empresariales.

De esta manera, el SGI integra en una única plataforma todos los procesos críticos de la organización.

3. PLANIFICACIÓN DEL PROYECTO

3.1 Metodología de Trabajo

Para el desarrollo del Sistema de Gestión Integrado (SGI) se utilizó una metodología de trabajo personal basada en principios ágiles, permitiendo avanzar de manera progresiva y adaptarse a los desafíos que surgieron durante el proyecto.

Claude me ayudó muchísimo a practicar con código y a resolver dudas durante el desarrollo. También me permitió comprender mejor los errores que iba cometiendo y aprender cómo evitarlos en el futuro, por ejemplo en la implementación de las cards de nuevo producto y nuevo cliente, que comparten la misma interfaz.

Para mantener un orden en las tareas y distribuir el trabajo de manera eficiente, utilicé ClickUp como herramienta de organización. Allí fui registrando las actividades pendientes, los avances realizados y los objetivos de cada etapa, lo que me permitió tener una visión clara del progreso del proyecto.

El enfoque iterativo utilizado facilitó la incorporación de mejoras continuas, la resolución de inconvenientes técnicos y la validación constante de los requerimientos establecidos para el sistema. Además, Claude fue de gran ayuda en la etapa de diseño, ya que permitió complementar ideas y definir una propuesta visual más clara, especialmente porque no contaba con tanta creatividad para esa parte del proyecto.

3.2 Organización del Proyecto

El proyecto fue organizado en distintas etapas de trabajo con el objetivo de mantener una estructura clara durante todo el desarrollo.

La primera etapa estuvo orientada a la investigación y aprendizaje de las tecnologías necesarias para construir la aplicación. Durante este período se estudiaron conceptos relacionados con React, FastAPI, PostgreSQL, Supabase y autenticación mediante JWT. Gran parte de esta investigación se apoyó en los cursos de Midudev, que me ayudaron aproximadamente en un 70% del proceso de aprendizaje, junto con la documentación de W3Schools y DataCamp. Además, Claude me ayudó a practicar con código y a reforzar conceptos durante esta etapa.

Posteriormente se realizó el análisis de requerimientos y el diseño general del sistema. En esta etapa se definieron las funcionalidades principales, los módulos que formarían parte de la aplicación y la estructura general de la base de datos.

Una vez finalizada la etapa de planificación comenzó el desarrollo simultáneo del frontend y backend. En esta etapa, FastAPI me ayudó muchísimo con todo lo relacionado con el backend, mientras que Supabase facilitó gran parte del trabajo con

la base de datos y la autenticación. Por otra parte, en el frontend Claude me ayudó muchísimo a resolver los errores que iba encontrando durante la implementación.

Finalmente, como prueba, se creó un sujeto de prueba con rol de vendedor para validar que pudiera crear productos, registrar un nuevo cliente, ajustar el stock y generar su propio usuario.

3.3 Cronograma de Actividades

Para organizar el desarrollo del sistema se elaboró un cronograma que permitió distribuir adecuadamente las tareas a lo largo de todo el proyecto.

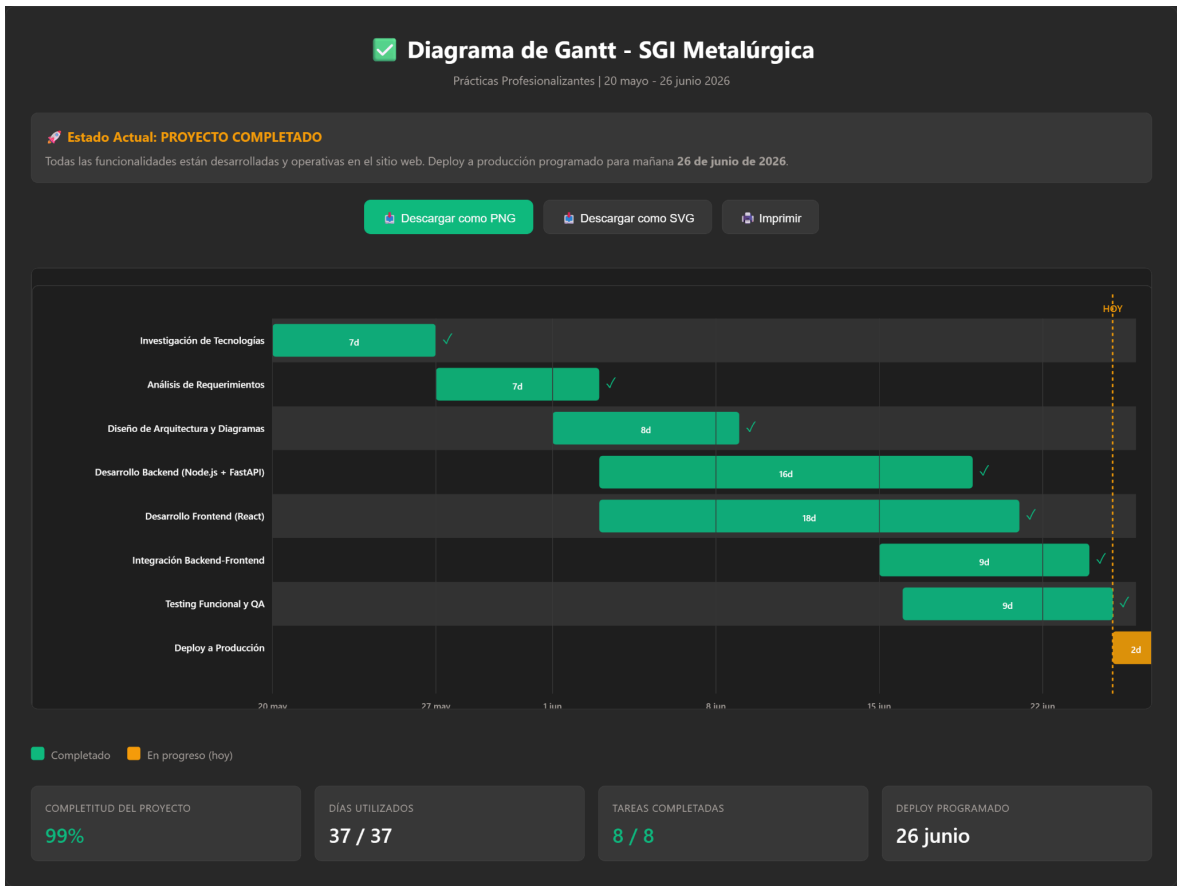
Las actividades fueron divididas en diferentes fases:

Fase	Descripción
Investigación y aprendizaje	Estudio de las tecnologías, herramientas y conceptos necesarios para el desarrollo del sistema, incluyendo React, FastAPI, PostgreSQL, Supabase y autenticación JWT.
Análisis de requerimientos	Relevamiento de las necesidades del proyecto y definición de los objetivos funcionales y técnicos de la aplicación.
Diseño del sistema	Diseño de la arquitectura general, la estructura de la base de datos y la organización de los módulos del frontend y backend.
Desarrollo backend	Implementación de la API, la lógica de negocio y el acceso a datos mediante FastAPI y PostgreSQL.
Desarrollo frontend	Construcción de la interfaz de usuario y de la experiencia visual utilizando React.
Integración	Conexión entre frontend, backend y base de datos para asegurar el correcto intercambio de información.
Pruebas y correcciones	Realización de pruebas funcionales, detección de errores y aplicación de correcciones para mejorar la estabilidad del sistema.
Documentación	Elaboración de la carpeta técnica y de la documentación final del proyecto.

3.4 Diagrama de Gantt

Para representar visualmente la distribución temporal de las actividades se desarrolló un **Diagrama de Gantt**.

Este diagrama permitió identificar dependencias entre tareas, establecer prioridades y controlar el progreso de cada fase del proyecto.



3.5 Gestión de Riesgos

Durante la organización y desarrollo del proyecto se identificaron distintos riesgos que podían afectar el cumplimiento de las etapas planificadas.

Aprendizaje de Nuevas Tecnologías

Uno de los principales riesgos estuvo relacionado con la etapa de investigación y aprendizaje, ya que varias de las tecnologías utilizadas eran nuevas para mí. Esto podía generar demoras en las primeras fases del proyecto y dificultades al momento de implementar funcionalidades más complejas.

Para reducir este riesgo se destinó tiempo específico al estudio de documentación oficial, ejemplos prácticos y pruebas iniciales antes de avanzar con el desarrollo de cada módulo.

Problemas de Integración

Otro riesgo importante surgió durante la etapa de integración entre frontend, backend y base de datos. Al trabajar en paralelo sobre distintas partes del sistema, existía la posibilidad de que aparecieran errores de comunicación entre los módulos.

Para minimizar este problema se realizaron pruebas de integración de manera continua a medida que se desarrollaban las funcionalidades, en lugar de dejar la validación para el final del proyecto.

Errores en el Diseño de la Base de Datos

Durante la etapa de diseño también existía el riesgo de definir incorrectamente la estructura de la base de datos, lo que podía provocar inconsistencias en la información o complicaciones en el funcionamiento general del sistema.

Para evitarlo se analizó previamente la relación entre las entidades del sistema y se revisó la estructura de los datos antes de comenzar con la implementación.

Problemas de Tiempo

La distribución del trabajo en distintas etapas exigía una buena administración del tiempo disponible. Debido a que el proyecto incluía investigación, diseño, desarrollo, pruebas y documentación, existía el riesgo de que alguna fase se retrasara y afectara al resto del cronograma.

Para reducir este riesgo se priorizaron primero las funcionalidades principales y luego se incorporaron mejoras adicionales según el avance del proyecto.

Errores Durante las Pruebas

En la etapa de testing podían aparecer fallas funcionales o errores no detectados en fases anteriores. Esto representaba un riesgo importante, ya que podía afectar la estabilidad del sistema al momento de la integración final.

Para afrontarlo se realizaron pruebas frecuentes sobre cada módulo desarrollado, permitiendo corregir errores de forma temprana y asegurar un funcionamiento más estable.

3.6 Seguimiento y Control del Proyecto

El seguimiento del proyecto se realizó de forma continua mediante la revisión diaria y semanal de los avances alcanzados en cada etapa.

Durante el desarrollo se verificó constantemente el cumplimiento de los requerimientos funcionales definidos en la fase de análisis. Cada módulo implementado fue probado individualmente antes de avanzar hacia nuevas funcionalidades.

Las tareas completadas fueron comparadas con la planificación inicial para identificar posibles desvíos y realizar los ajustes necesarios.

Asimismo, se llevaron a cabo pruebas funcionales frecuentes para detectar errores de manera temprana y garantizar la estabilidad del sistema.

Para la organización de las tareas y el seguimiento de los errores se utilizó ClickUp, herramienta que resultó de gran ayuda para mantener un control ordenado del trabajo pendiente y de los problemas detectados durante el desarrollo. Por otra parte, Claude fue de gran utilidad cuando surgían errores que no lograba comprender completamente, ya que me ayudaba a analizarlos, entender su causa y evitar repetirlos en futuras implementaciones.

Debido al tiempo limitado disponible para finalizar el proyecto, fue necesario priorizar las tareas más importantes y mantener una planificación constante para poder avanzar de manera eficiente.

4. ARQUITECTURA GENERAL DEL SISTEMA

4.1 Arquitectura Cliente – Servidor

El Sistema de Gestión Integrado (SGI) fue desarrollado utilizando una arquitectura cliente-servidor, uno de los modelos más utilizados actualmente en aplicaciones web empresariales.

En este modelo, la aplicación se divide en dos partes principales: el cliente (frontend) y el servidor (backend). Ambas capas trabajan de forma independiente pero coordinada mediante solicitudes HTTP.

El frontend es el encargado de interactuar con el usuario, mostrar información y capturar acciones realizadas desde la interfaz gráfica. Por otro lado, el backend se

encarga de procesar las solicitudes, aplicar las reglas de negocio, validar los datos y comunicarse con la base de datos.

La principal ventaja de esta arquitectura es la separación de responsabilidades, permitiendo que cada componente evolucione de forma independiente y facilitando el mantenimiento del sistema.

De manera simplificada, **el flujo de trabajo funciona de la siguiente manera:**

1. El usuario realiza una acción desde el navegador.
2. El frontend envía una solicitud HTTP al backend.
3. El backend procesa la solicitud.
4. Se consulta o modifica información en la base de datos.
5. El backend devuelve una respuesta.
6. El frontend actualiza la interfaz con los datos recibidos.

Esta estructura permitió construir una aplicación escalable, organizada y preparada para futuras ampliaciones.

4.2 Arquitectura Frontend

La capa frontend fue desarrollada utilizando React, una biblioteca JavaScript moderna orientada a la construcción de interfaces de usuario dinámicas e interactivas.

La aplicación fue diseñada siguiendo una arquitectura basada en componentes reutilizables. Cada pantalla y funcionalidad se encuentra dividida en componentes independientes que facilitan la reutilización del código y el mantenimiento futuro.

Entre las principales responsabilidades del frontend se encuentran:

- Gestión de la interfaz de usuario.
- Navegación entre módulos.
- Validación inicial de formularios.
- Consumo de APIs.
- Visualización de reportes.
- Gestión de estados de la aplicación.

- Manejo de autenticación del usuario.

La utilización de React permitió implementar una experiencia de usuario moderna, con actualizaciones dinámicas de información sin necesidad de recargar la página completa.

Además, la estructura basada en componentes facilitó el desarrollo de elementos reutilizables como tablas, formularios, tarjetas informativas, modales y paneles de estadísticas.

Asimismo, Claude fue de gran ayuda durante el desarrollo, especialmente en la propuesta de iconos y en el diseño visual de las interfaces de cada sección, lo que permitió mejorar la presentación general del sistema y lograr una apariencia más ordenada, intuitiva y profesional.

Principales módulos del frontend

- Inicio de sesión.
- Dashboard principal.
- Gestión de usuarios.
- Gestión de clientes.
- Gestión de productos.
- Gestión de stock.
- Gestión de pedidos.
- Monitor de producción.
- Reportes e indicadores.

4.3 Arquitectura Backend

La capa backend fue desarrollada utilizando FastAPI, un framework moderno de Python especializado en la construcción de APIs REST de alto rendimiento.

El backend actúa como núcleo lógico del sistema y es responsable de procesar todas las operaciones realizadas por los usuarios.

Entre sus funciones principales se encuentran:

- Autenticación y autorización.

- Validación de datos.
- Gestión de usuarios.
- Gestión de clientes.
- Gestión de productos.
- Gestión de stock.
- Gestión de pedidos.
- Generación de reportes.
- Comunicación con la base de datos.

La API fue diseñada siguiendo principios REST, permitiendo que cada módulo disponga de endpoints específicos para realizar operaciones de consulta, creación, actualización y eliminación de datos.

La utilización de FastAPI aportó ventajas importantes:

- Alto rendimiento.
- Validación automática de datos.
- Documentación automática de endpoints.
- Fácil mantenimiento.
- Integración sencilla con PostgreSQL.

Esta arquitectura permitió mantener separada la lógica de negocio de la interfaz de usuario, favoreciendo la escalabilidad y la organización general del proyecto.

4.4 Arquitectura de Base de Datos

Para el almacenamiento de información se utilizó PostgreSQL a través de la plataforma Supabase.

La base de datos constituye el repositorio central de toda la información utilizada por el sistema.

Dentro de ella se almacenan:

- Usuarios.

- Roles.
- Clientes.
- Productos.
- Componentes.
- Movimientos de stock.
- Pedidos.
- Detalles de pedidos.
- Estados de producción.
- Información de reportes.

La elección de PostgreSQL se debió a su confiabilidad, rendimiento, capacidad para gestionar relaciones complejas entre entidades, además de su facilidad de uso en un lapso de tiempo tan corto.

Supabase permitió simplificar la administración de la base de datos al proporcionar una infraestructura lista para utilizar, con acceso remoto, seguridad integrada y herramientas de gestión visual.

La estructura relacional implementada garantiza la integridad de los datos y evita inconsistencias entre los distintos módulos del sistema.

4.5 Flujo General de Información

El flujo de información dentro del sistema sigue un proceso claramente definido.

Cuando un usuario inicia sesión, el frontend envía sus credenciales al backend.

El backend valida la información recibida y, si los datos son correctos, genera un token JWT que será utilizado durante toda la sesión para identificar al usuario.

Posteriormente, cada vez que el usuario realiza una acción, el frontend envía una solicitud autenticada al backend utilizando dicho token.

El backend verifica los permisos correspondientes y ejecuta la operación solicitada.

Finalmente, la información es almacenada o recuperada desde PostgreSQL y enviada nuevamente al frontend para su visualización.

Este mecanismo garantiza:

- Seguridad.
- Integridad de los datos.
- Control de acceso.
- Trazabilidad de las operaciones.

Cabe mencionar que en esta etapa Claude me ayudó bastante, ya que yo contaba con algunos conocimientos previos, pero su apoyo fue muy útil para organizar mejor las ideas, redactar con mayor claridad y estructurar correctamente esta parte del proyecto.

4.6 Integración de Componentes

La integración entre los distintos componentes del sistema constituye uno de los aspectos más importantes de la arquitectura implementada.

El frontend, el backend y la base de datos trabajan de manera coordinada para proporcionar una experiencia fluida al usuario.

React se comunica con FastAPI mediante solicitudes HTTP utilizando endpoints específicos para cada módulo.

FastAPI procesa la información recibida y realiza las operaciones necesarias sobre PostgreSQL a través de Supabase.

Una vez completada la operación, la respuesta es enviada nuevamente al frontend para actualizar la interfaz de usuario.

Esta comunicación permanente permite mantener la información sincronizada en tiempo real y garantizar el correcto funcionamiento de todas las funcionalidades del sistema.

La arquitectura implementada favorece la modularidad, el mantenimiento y la posibilidad de incorporar nuevas características en futuras versiones sin afectar el funcionamiento general de la aplicación.

4.7 Beneficios de la Arquitectura Implementada

La arquitectura seleccionada aportó múltiples beneficios durante el desarrollo y funcionamiento del sistema:

- Separación clara entre frontend y backend.
- Mayor facilidad de mantenimiento.
- Escalabilidad para futuras mejoras.
- Mejor organización del código fuente.
- Mayor seguridad mediante autenticación JWT.
- Integración eficiente con PostgreSQL.
- Desarrollo modular y reutilizable.
- Capacidad para incorporar nuevos módulos sin rediseñar la aplicación.

Gracias a esta arquitectura fue posible desarrollar una solución robusta, moderna y preparada para acompañar el crecimiento futuro del software.

5. TECNOLOGÍAS UTILIZADAS

5.1 React

React se utilizó para desarrollar el frontend del sistema. Esta biblioteca permitió crear interfaces dinámicas y reutilizables, facilitando la organización y el mantenimiento del código. Gracias a su enfoque basado en componentes, fue posible estructurar la aplicación de manera clara y modular.

5.2 FastAPI

FastAPI fue la tecnología elegida para el backend. Permitted desarrollar una API rápida, segura y fácil de mantener, además de facilitar la comunicación entre el frontend y la base de datos. También aportó ventajas como la validación automática de datos y la generación de documentación de los endpoints.

5.3 PostgreSQL

PostgreSQL se utilizó como sistema gestor de base de datos. Esta tecnología permitió almacenar la información de forma segura, ordenada y escalable. Su capacidad para manejar relaciones entre tablas resultó fundamental para la estructura del sistema.

5.4 Supabase

Supabase se utilizó para administrar y acceder a la base de datos. Su uso simplificó la configuración, la gestión de tablas y el acceso remoto a la información. Además, facilitó el trabajo con PostgreSQL mediante una interfaz visual práctica y accesible.

5.5 JSON Web Token (JWT)

JWT se utilizó para implementar la autenticación y autorización de usuarios, permitiendo controlar el acceso según el rol de cada uno. Esta tecnología ayudó a reforzar la seguridad del sistema y a proteger las rutas sensibles de la aplicación.

5.6 Visual Studio Code

Visual Studio Code fue el entorno de desarrollo principal. Facilitó el trabajo en frontend y backend gracias a sus herramientas de edición, integración con Git y terminal incorporada. También permitió organizar mejor el proyecto y agilizar el proceso de desarrollo.

5.7 Claude

Claude se utilizó como herramienta de apoyo durante el desarrollo. Fue útil para resolver errores, comprender conceptos nuevos y mejorar la implementación de distintas funcionalidades. También ayudó en la generación de ideas, la organización del código y la selección de recursos visuales.

5.8 Recursos de Aprendizaje

Durante el proyecto también se consultaron cursos y documentación oficial de tecnologías como React, FastAPI, PostgreSQL y Supabase, además de otras plataformas de aprendizaje.

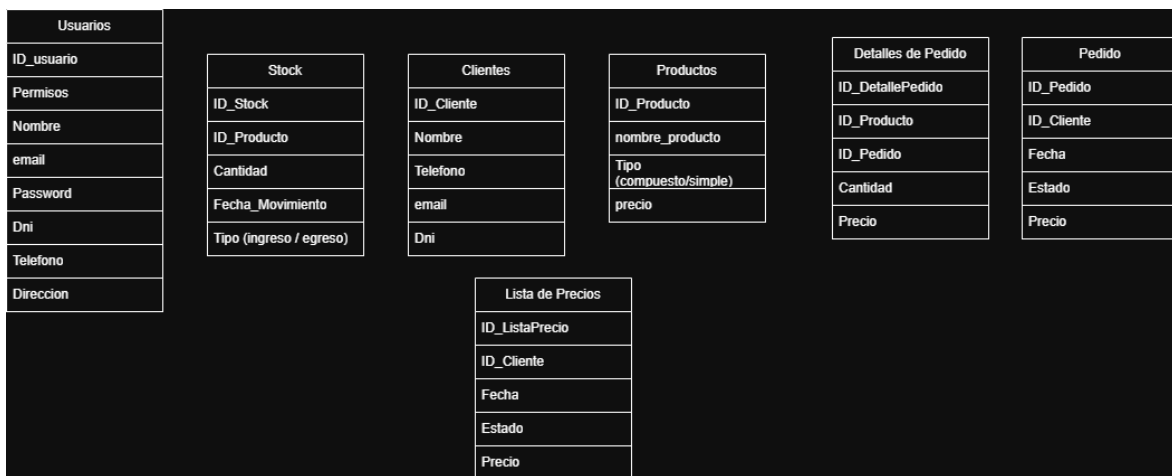
6. Diseño de la Base de Datos

6.1 Modelo Entidad-Relación

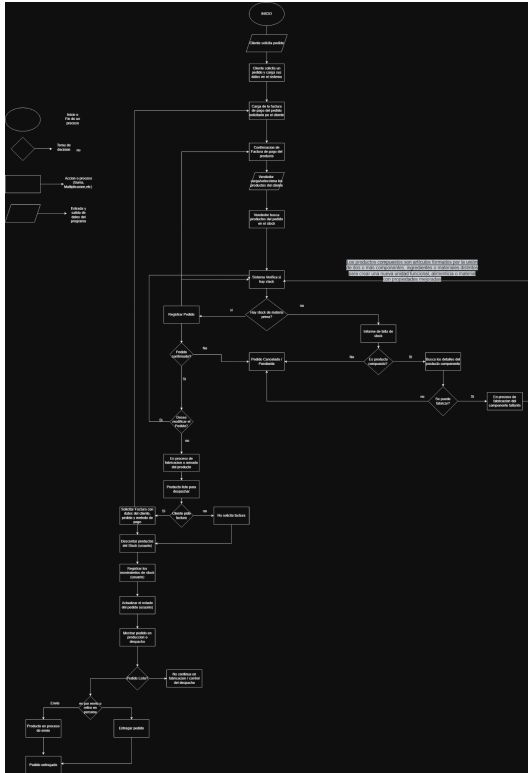
Antes de comenzar con la implementación física de la base de datos, se realizó un modelo Entidad-Relación (DER) con el objetivo de representar gráficamente las entidades principales del sistema y las relaciones existentes entre ellas.

Este modelo permitió identificar cómo interactúan los distintos módulos del SGI, facilitando el diseño de una estructura organizada y escalable. Gracias al DER fue posible definir las tablas necesarias, sus atributos y las relaciones que garantizan la integridad de la información almacenada.

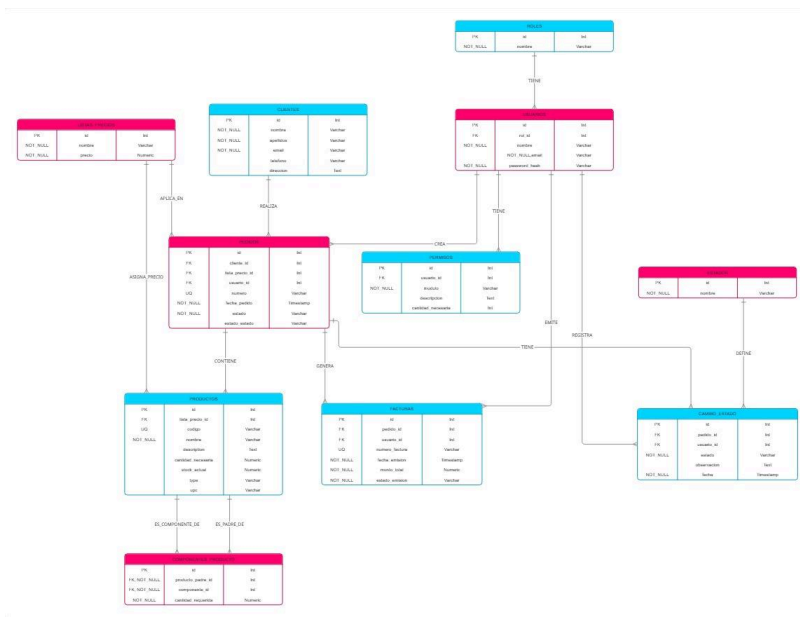
Primer boceto del DER (Algunas de estas tablas las use para la arquitectura de la base de datos):



2do boceto de DER tipo CECH (modelo usuario):



DER FINAL DEL SGI:



6.2 Tabla Usuarios

La tabla Usuarios almacena la información de las personas que pueden acceder al sistema. Cada registro contiene los datos necesarios para identificar al usuario y controlar su acceso a las distintas funcionalidades.

Esta tabla permite administrar cuentas, autenticar usuarios y registrar las acciones realizadas dentro de la plataforma.

6.3 Tabla Roles

La tabla Roles define los distintos niveles de acceso disponibles dentro del sistema.

Cada usuario se encuentra asociado a un rol específico, el cual determina las funcionalidades y módulos a los que puede acceder. Esta estructura permite implementar un esquema de permisos organizado y mejorar la seguridad de la aplicación.

6.4 Tabla Clientes

La tabla Clientes almacena la información de las personas o empresas que realizan operaciones con la organización.

Su función principal es centralizar los datos comerciales necesarios para la generación de pedidos, consultas y reportes, facilitando la gestión de las relaciones con los clientes.

6.5 Tabla Productos

La tabla Productos contiene el catálogo general de artículos administrados por el sistema.

En ella se registran tanto productos simples como productos compuestos, incluyendo la información necesaria para su identificación, clasificación y control dentro de los procesos operativos de la empresa.

6.6 Tabla Componentes

La tabla Componentes permite gestionar la composición de los productos compuestos.

Mediante esta estructura se establece qué materiales o productos forman parte de un producto final, así como las cantidades necesarias para su fabricación. Esta información resulta fundamental para el cálculo de stock y la planificación de la producción.

6.7 Tabla Stock

La tabla Stock almacena la información relacionada con las existencias disponibles de cada producto.

Su principal objetivo es permitir el control actualizado del inventario, facilitando la consulta de cantidades disponibles y la detección de niveles críticos de stock.

6.8 Tabla Pedidos

La tabla Pedidos registra las órdenes realizadas por los clientes.

Cada pedido contiene información relacionada con el cliente, la fecha de creación, el estado actual y demás datos necesarios para realizar el seguimiento completo de la operación desde su inicio hasta su finalización.

6.9 Tabla Detalle de Pedidos

La tabla Detalle de Pedidos almacena los productos incluidos en cada pedido.

Su función es relacionar cada pedido con los artículos solicitados, registrando cantidades, precios y demás datos necesarios para la correcta gestión comercial y operativa.

6.10 Tabla Movimientos de Stock

La tabla Movimientos de Stock registra todas las operaciones que generan cambios en el inventario.

Cada movimiento queda almacenado con información sobre el producto afectado, la fecha de la operación y el tipo de movimiento realizado, permitiendo mantener la trazabilidad completa de las existencias.

6.11 Relaciones e Integridad Referencial

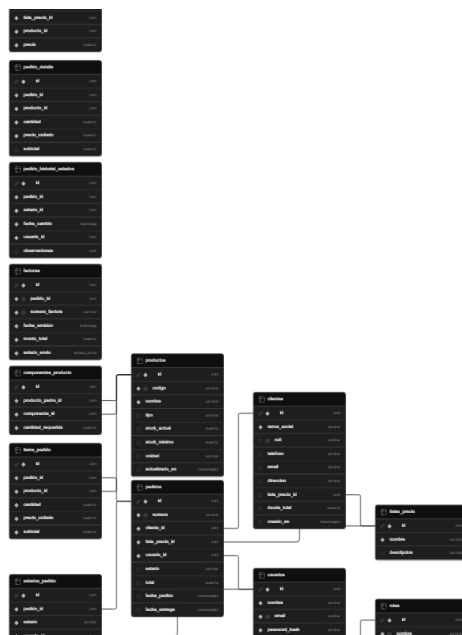
Las distintas tablas del sistema se encuentran relacionadas mediante claves primarias y claves foráneas que garantizan la integridad de la información.

Estas relaciones permiten mantener la consistencia de los datos, evitando registros inválidos y asegurando que todas las operaciones realizadas dentro del sistema respeten las reglas establecidas por el modelo de negocio.

La implementación de integridad referencial contribuye a la confiabilidad general del sistema y facilita el correcto funcionamiento de todos sus módulos.

Diagrama extraído de supabase con todas las tablas anteriormente mencionadas:

(también visible en supabase)



7. Desarrollo Backend

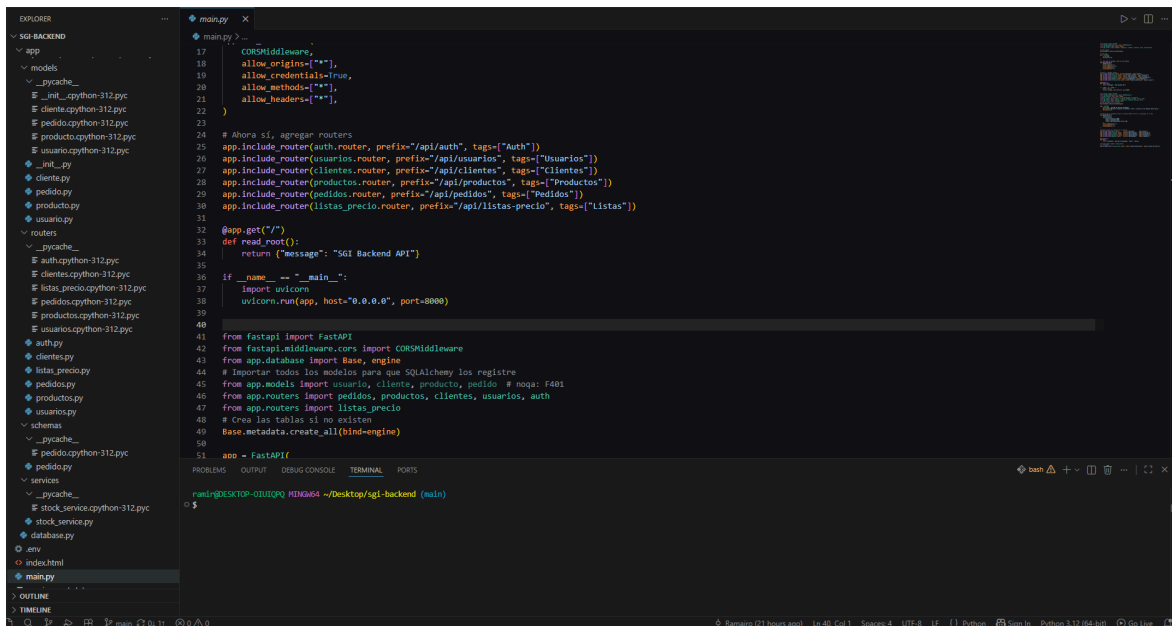
7.1 Introducción al Backend

El backend del Sistema de Gestión Integrado fue desarrollado utilizando FastAPI, un framework moderno de Python orientado a la creación de APIs REST.

Esta capa es responsable de procesar las solicitudes realizadas por los usuarios, aplicar las reglas de negocio, validar la información recibida y gestionar la comunicación con la base de datos.

Gracias a esta arquitectura fue posible separar la lógica del sistema de la interfaz de usuario, facilitando el mantenimiento y la escalabilidad de la aplicación.

Visualización de el Backend del archivo main



```
17 from fastapi import FastAPI
18 from fastapi.middleware.cors import CORSMiddleware
19 from app.database import Base, engine
20 from app.models import usuario, cliente, producto, pedido
21 from app.routers import auth, usuarios, clientes, productos, pedidos, listasPrecio
22
23 # Ahora sí, agregar routers
24 app.include_router(auth.router, prefix="/api/auth", tags=["Auth"])
25 app.include_router(usuarios.router, prefix="/api/usuarios", tags=["Usuarios"])
26 app.include_router(clientes.router, prefix="/api/clientes", tags=["Clientes"])
27 app.include_router(productos.router, prefix="/api/productos", tags=["Productos"])
28 app.include_router(pedidos.router, prefix="/api/pedidos", tags=["Pedidos"])
29 app.include_router(listasPrecio.router, prefix="/api/listas-precio", tags=["Listas"])
30
31 @app.get("/")
32 def read_root():
33     return {"message": "SGI Backend API"}
34
35 if __name__ == "__main__":
36     import uvicorn
37     uvicorn.run(app, host="0.0.0.0", port=8000)
38
39
40 from fastapi import FastAPI
41 from fastapi.middleware.cors import CORSMiddleware
42 from app.database import Base, engine
43 # Importar todos los modelos para que SQLAlchemy los registre
44 from app.models import usuario, cliente, producto, pedido
45 from app.routers import auth, usuarios, clientes, productos, pedidos, listasPrecio
46 # Crea las tablas si no existen
47 Base.metadata.create_all(bind=engine)
48
49 app = FastAPI()
50
51 # Incluye routers
52 app.include_router(auth.router, prefix="/api/auth", tags=["Auth"])
53 app.include_router(usuarios.router, prefix="/api/usuarios", tags=["Usuarios"])
54 app.include_router(clientes.router, prefix="/api/clientes", tags=["Clientes"])
55 app.include_router(productos.router, prefix="/api/productos", tags=["Productos"])
56 app.include_router(pedidos.router, prefix="/api/pedidos", tags=["Pedidos"])
57 app.include_router(listasPrecio.router, prefix="/api/listas-precio", tags=["Listas"])
58
59 @app.get("/")
60 def read_root():
61     return {"message": "SGI Backend API"}
62
63 if __name__ == "__main__":
64     import uvicorn
65     uvicorn.run(app, host="0.0.0.0", port=8000)
```

7.2 Estructura del Proyecto

El backend fue organizado de manera modular con el objetivo de mejorar la legibilidad del código y facilitar futuras ampliaciones.

La estructura se dividió en distintos componentes encargados de funciones específicas, como autenticación, gestión de usuarios, clientes, productos, stock y pedidos.

Esta organización permitió mantener una clara separación de responsabilidades y simplificó el desarrollo de nuevas funcionalidades.

7.3 Configuración Inicial

La configuración inicial del backend incluyó la instalación de dependencias, definición de variables de entorno y conexión con la base de datos.

También se configuraron aspectos relacionados con la seguridad, autenticación y comunicación entre el servidor y el frontend.

Esta etapa permitió establecer la base necesaria para el desarrollo de todos los módulos posteriores.

7.4 Gestión de Autenticación

La autenticación fue implementada mediante JSON Web Tokens (JWT).

Cuando un usuario inicia sesión, el sistema valida sus credenciales y genera un token que será utilizado para identificarlo durante toda la sesión.

Este mecanismo permite mantener la seguridad de la aplicación y controlar el acceso a los distintos recursos del sistema.

7.5 Gestión de Usuarios

El módulo de usuarios permite administrar las cuentas registradas dentro del sistema.

A través de este módulo es posible crear nuevos usuarios, modificar información existente, asignar roles y controlar el acceso a las distintas funcionalidades disponibles.

Su implementación contribuye a mantener una gestión organizada y segura de los usuarios de la aplicación.

7.6 Gestión de Clientes

El módulo de clientes fue desarrollado para centralizar la información comercial de las personas y empresas que operan con la organización.

Permite registrar nuevos clientes, consultar datos existentes y mantener actualizada la información necesaria para la gestión de pedidos y reportes.

7.7 Gestión de Productos

El módulo de productos administra el catálogo general utilizado por la empresa.

Su funcionalidad permite registrar productos simples y compuestos, actualizar información relevante y mantener un control organizado sobre los artículos gestionados dentro del sistema.

Este módulo constituye uno de los componentes centrales de la aplicación debido a su relación con el stock, los pedidos y la producción.

7.8 Gestión de Stock

El módulo de stock fue diseñado para controlar las existencias disponibles de cada producto.

A través de este componente se registran ingresos, egresos y ajustes de inventario, permitiendo mantener información actualizada sobre la disponibilidad de los distintos artículos.

Su implementación facilita la toma de decisiones y reduce el riesgo de errores operativos.

7.9 Gestión de Pedidos

El módulo de pedidos permite registrar y administrar las órdenes realizadas por los clientes.

Durante su procesamiento se validan datos relevantes y se realiza el seguimiento de los distintos estados por los que atraviesa cada pedido.

Esta funcionalidad facilita el control de las operaciones comerciales y mejora la trazabilidad de los procesos.

7.10 Gestión de Producción

El módulo de producción fue desarrollado para realizar el seguimiento de las actividades relacionadas con la fabricación de productos.

Su principal objetivo es proporcionar información sobre el estado de avance de los pedidos y facilitar el control de los procesos productivos dentro de la organización.

7.11 Gestión de Reportes

El sistema incorpora herramientas destinadas a la generación de reportes e indicadores.

Estos reportes permiten visualizar información relacionada con clientes, productos, stock y pedidos, facilitando el análisis de datos y la toma de decisiones.

La centralización de esta información contribuye a mejorar el control operativo de la empresa.

7.12 Manejo de Excepciones

Durante el desarrollo se implementaron mecanismos de manejo de excepciones con el objetivo de detectar y controlar posibles errores del sistema.

Estas validaciones permiten generar respuestas adecuadas ante situaciones inesperadas, evitando fallos críticos y mejorando la experiencia del usuario.

Además, facilitan las tareas de mantenimiento y depuración de la aplicación.

7.13 Validaciones y Seguridad

La aplicación incorpora diversas medidas de validación y seguridad para garantizar la integridad de la información.

Entre ellas se incluyen controles de acceso, validación de datos recibidos, autenticación mediante JWT y restricciones de permisos según el rol del usuario.

Estas medidas contribuyen a proteger los recursos del sistema y evitar accesos no autorizados.

7.14 API REST Implementada

La comunicación entre el frontend y el backend se realiza mediante una API REST desarrollada con FastAPI.

Esta API expone distintos endpoints que permiten realizar operaciones de consulta, creación, modificación y eliminación de información.

Gracias a este enfoque, la aplicación mantiene una arquitectura desacoplada y facilita la integración entre los distintos componentes del sistema.

La implementación de una API REST también favorece futuras ampliaciones y la posible integración con otros sistemas externos.

API REST - DESARROLLADA CON FAST API:

SGI - Sistema de Gestión Integrado 1.0.0 OAS 3.1		
openapi.json		
API para la gestión de pedidos, stock y clientes de una empresa metalúrgica.		
Auth ^		
POST	/api/auth/login	Login
POST	/api/auth/seed-admin	Crear Admin
Usuarios ^		
GET	/api/usuarios/	Listar Usuarios
POST	/api/usuarios/	Crear Usuario
GET	/api/usuarios/roles	Listar Roles
GET	/api/usuarios/{id}	Obtener Usuario
PUT	/api/usuarios/{id}	Editar Usuario
PATCH	/api/usuarios/{id}/estado	Cambiar Estado Usuario
Clientes ^		
GET	/api/clientes/	Listar Clientes
POST	/api/clientes/	Crear Cliente

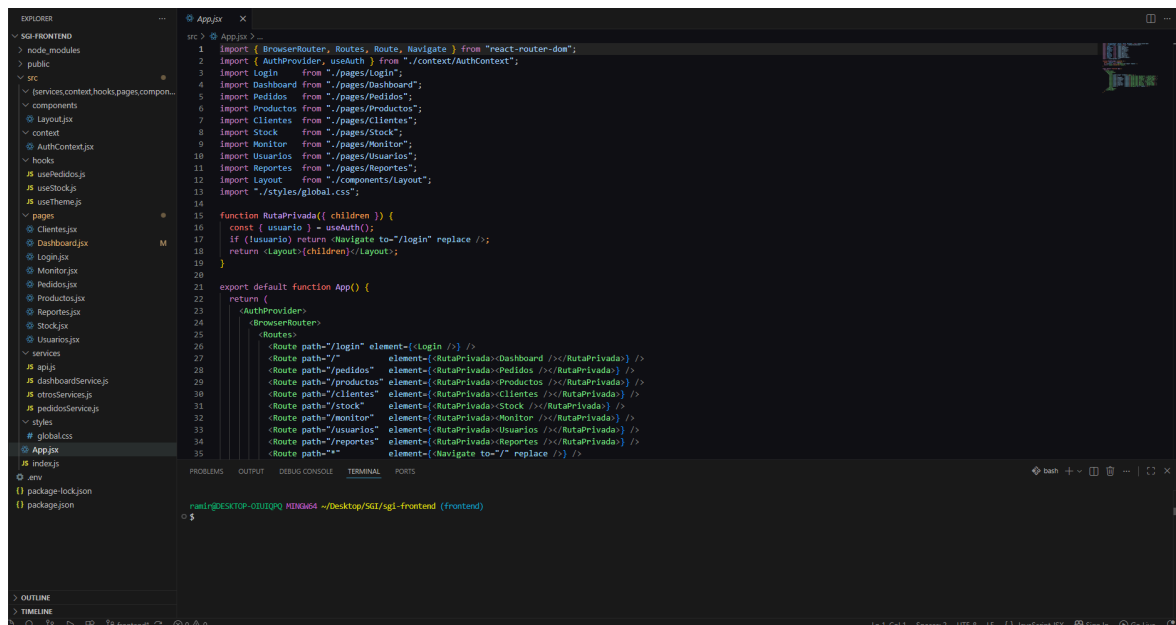
8. Desarrollo Frontend

8.1 Introducción al Frontend

El frontend del Sistema de Gestión Integrado fue desarrollado utilizando React, una biblioteca de JavaScript orientada a la construcción de interfaces dinámicas y reutilizables.

Su función principal consiste en permitir la interacción entre los usuarios y el sistema mediante una interfaz intuitiva, moderna y fácil de utilizar.

Visualización de el Frontend del archivo App



```
src > App.js
1 import { BrowserRouter, Routes, Route, Navigate } from "react-router-dom";
2 import { AuthProvider, useAuth } from "../context/AuthContext";
3 import Login from "../pages/Login";
4 import Dashboard from "../pages/Dashboard";
5 import Pedidos from "../pages/Pedidos";
6 import Productos from "../pages/Productos";
7 import Clientes from "../pages/Clientes";
8 import Stock from "../pages/Stock";
9 import Monitor from "../pages/Monitor";
10 import Usuarios from "../pages/Usuarios";
11 import Reportes from "../pages/Reportes";
12 import Layout from "../components/Layout";
13 import "../styles/global.css";
14
15 function RutaPrivada({ children }) {
16   const { usuario } = useAuth();
17   if (!usuario) return <Navigate to="/login" replace />;
18   return <Layout>{children}</Layout>;
19 }
20
21 export default function App() {
22   return (
23     <AuthProvider>
24       <BrowserRouter>
25         <Routes>
26           <Route path="/login" element=<Login /> />
27           <Route path="/" element=<RutaPrivada><Dashboard /></RutaPrivada> />
28           <Route path="/pedidos" element=<RutaPrivada><Pedidos /></RutaPrivada> />
29           <Route path="/productos" element=<RutaPrivada><Productos /></RutaPrivada> />
30           <Route path="/clientes" element=<RutaPrivada><Clientes /></RutaPrivada> />
31           <Route path="/stock" element=<RutaPrivada><Stock /></RutaPrivada> />
32           <Route path="/monitor" element=<RutaPrivada><Monitor /></RutaPrivada> />
33           <Route path="/usuarios" element=<RutaPrivada><Usuarios /></RutaPrivada> />
34           <Route path="/reportes" element=<RutaPrivada><Reportes /></RutaPrivada> />
35           <Route path="/" element=<Navigate to="/" replace /> />
36         </Routes>
37       </BrowserRouter>
38     </AuthProvider>
39   );
40 }
```

8.2 Arquitectura de Componentes

La aplicación fue diseñada siguiendo una arquitectura basada en componentes, permitiendo dividir la interfaz en elementos independientes y reutilizables.

Esta metodología facilita el mantenimiento del código, mejora la organización del proyecto y permite reutilizar funcionalidades en distintas secciones del sistema.

8.3 Sistema de Navegación

La navegación entre las diferentes pantallas fue implementada mediante rutas que permiten acceder a cada módulo de manera rápida y organizada.

Este sistema facilita el desplazamiento dentro de la aplicación y mejora la experiencia de usuario al mantener una estructura clara y consistente.

8.4 Gestión de Estados

La gestión de estados permite administrar la información utilizada por los distintos componentes de la aplicación.

Su implementación facilita la actualización dinámica de datos y asegura que la interfaz refleje correctamente los cambios realizados por los usuarios sin necesidad de recargar la página.

8.5 Pantalla de Login

La pantalla de inicio de sesión constituye el punto de acceso principal al sistema.

Desde esta interfaz los usuarios ingresan sus credenciales para autenticarse y acceder a las funcionalidades correspondientes según su rol dentro de la organización.

8.6 Dashboard Principal

El dashboard funciona como la pantalla principal del sistema una vez iniciada la sesión.

Su objetivo es presentar información relevante de forma resumida, permitiendo a los usuarios acceder rápidamente a los distintos módulos y visualizar indicadores generales del sistema.

8.7 Módulo de Usuarios

El módulo de usuarios permite administrar las cuentas registradas dentro de la plataforma.

A través de esta sección es posible crear, modificar, consultar y gestionar los usuarios autorizados para utilizar el sistema.

8.8 Módulo de Clientes

El módulo de clientes centraliza la información comercial necesaria para la gestión de operaciones.

Permite registrar nuevos clientes, actualizar datos existentes y consultar información relevante para la generación de pedidos y reportes.

8.9 Módulo de Productos

El módulo de productos administra el catálogo general de artículos gestionados por la empresa.

Desde esta sección se pueden visualizar, registrar y modificar productos, manteniendo organizada la información utilizada por otros módulos del sistema.

8.10 Módulo de Stock

El módulo de stock permite consultar y controlar las existencias disponibles de cada producto.

Su implementación facilita el seguimiento del inventario y proporciona información actualizada para la toma de decisiones operativas.

8.11 Módulo de Pedidos

El módulo de pedidos permite registrar y administrar las órdenes realizadas por los clientes.

Además de almacenar la información correspondiente, facilita el seguimiento de cada pedido durante las distintas etapas de su ciclo de vida.

8.12 Módulo de Producción

El módulo de producción fue desarrollado para visualizar y controlar el estado de avance de los procesos productivos.

Esta herramienta permite realizar un seguimiento más eficiente de las tareas asociadas a la fabricación de productos y al cumplimiento de los pedidos.

8.13 Módulo de Reportes

El módulo de reportes proporciona acceso a información consolidada del sistema mediante gráficos, tablas e indicadores.

Su finalidad es facilitar el análisis de datos y apoyar la toma de decisiones dentro de la organización.

8.14 Componentes Reutilizables

Durante el desarrollo se implementaron componentes reutilizables para reducir la duplicación de código y mantener una apariencia uniforme en toda la aplicación.

Entre ellos se encuentran formularios, tablas, botones, modales, tarjetas informativas y elementos de navegación.

Esta estrategia permitió mejorar la mantenibilidad y escalabilidad del proyecto.

8.15 Hooks Personalizados

Para optimizar la lógica de la aplicación se utilizaron hooks personalizados que encapsulan funcionalidades reutilizables dentro del frontend.

Su implementación permitió centralizar procesos repetitivos, mejorar la organización del código y simplificar el mantenimiento de los distintos módulos del sistema.

9. Seguridad del Sistema

Apoyo durante la implementación (Aclaración)

Durante el desarrollo de los mecanismos de autenticación y autorización, fue necesario profundizar en conceptos de seguridad web que inicialmente no formaban parte de mis conocimientos previos. En esta etapa, la herramienta Claude resultó de gran ayuda para comprender el funcionamiento de los JSON Web Tokens (JWT), la gestión de sesiones, la protección de rutas y la implementación de permisos basados en roles. También porque no tenía el tiempo y no estaba tan informado sobre el lenguaje pero gracias a Claude, YouTube y documentación pude hacerlo lo mejor posible.

9.1 Autenticación JWT

La autenticación del sistema fue implementada mediante JSON Web Tokens (JWT), permitiendo identificar de forma segura a cada usuario durante su sesión.

Cuando un usuario inicia sesión, el backend valida sus credenciales y genera un token que será utilizado en las solicitudes posteriores para verificar su identidad.

Este mecanismo evita accesos no autorizados y mejora la seguridad general de la aplicación.

9.2 Autorización por Roles

El sistema implementa un esquema de autorización basado en roles, permitiendo asignar distintos niveles de acceso según las responsabilidades de cada usuario.

De esta manera, cada rol posee permisos específicos sobre los módulos y funcionalidades disponibles, garantizando que los usuarios únicamente puedan realizar las acciones correspondientes a su perfil.

Esta estrategia mejora el control interno y reduce riesgos asociados a accesos indebidos.

9.3 Protección de Rutas

Las rutas protegidas permiten restringir el acceso a determinadas secciones de la aplicación.

Antes de permitir el ingreso a un módulo, el sistema verifica que el usuario se encuentre autenticado y posea los permisos necesarios para acceder a dicho recurso.

Gracias a este mecanismo se evita que usuarios no autorizados accedan a información sensible o funciones restringidas.

9.4 Validación de Datos

Tanto el frontend como el backend incorporan procesos de validación de datos para garantizar que la información ingresada sea correcta antes de ser almacenada o procesada.

Estas validaciones permiten detectar errores, prevenir inconsistencias y mejorar la calidad de los datos registrados dentro del sistema.

Además, contribuyen a reducir posibles fallos durante la ejecución de las distintas funcionalidades.

9.5 Seguridad de la Información

La seguridad de la información fue considerada como un aspecto fundamental durante el desarrollo del proyecto.

Para proteger los datos almacenados se implementaron mecanismos de autenticación, control de acceso, validación de información y gestión segura de sesiones.

Estas medidas permiten preservar la confidencialidad, integridad y disponibilidad de la información, garantizando un funcionamiento confiable y seguro del Sistema de Gestión Integrado.

10. Testing y Control de Calidad

10.1 Estrategia de Testing

Durante el desarrollo del Sistema de Gestión Integrado se aplicó una estrategia de testing progresiva, basada en la verificación constante de cada funcionalidad implementada. A medida que se completaban los distintos módulos, se realizaban pruebas para comprobar su correcto funcionamiento antes de continuar con nuevas tareas.

Además, se verificó la correcta comunicación entre el frontend, el backend y la base de datos, permitiendo detectar errores de forma temprana y reducir problemas durante la integración final del sistema.

10.2 Pruebas de Login y Acceso de Usuarios

Se realizaron múltiples pruebas relacionadas con el inicio de sesión y la gestión de usuarios para garantizar el correcto funcionamiento de la autenticación.

Estas pruebas incluyeron la validación de credenciales correctas e incorrectas, el acceso según los distintos roles definidos y la verificación del funcionamiento de los tokens JWT generados durante la sesión.

Gracias a estas pruebas fue posible asegurar que únicamente los usuarios autorizados pudieran acceder a las funcionalidades correspondientes dentro del sistema.

10.3 Pruebas Funcionales

Las pruebas funcionales tuvieron como objetivo verificar que cada módulo cumpliera correctamente con los requerimientos definidos durante la etapa de análisis.

Se realizaron pruebas sobre la gestión de usuarios, clientes, productos, stock, pedidos, producción y reportes, comprobando que las operaciones principales pudieran ejecutarse correctamente y que la información se almacenara de manera adecuada.

Estas verificaciones permitieron confirmar que las funcionalidades desarrolladas respondían a las necesidades planteadas para el proyecto.

10.4 Pruebas de Integración

Las pruebas de integración se enfocan en verificar la correcta comunicación entre los distintos componentes del sistema.

Se comprobó el intercambio de información entre el frontend desarrollado en React, el backend implementado con FastAPI y la base de datos PostgreSQL administrada mediante Supabase.

Estas pruebas permitieron validar que los datos enviados por los usuarios fueran procesados correctamente y reflejados de forma consistente en toda la aplicación.

10.5 Resultados Obtenidos

Los resultados obtenidos durante el proceso de testing fueron satisfactorios, permitiendo verificar el correcto funcionamiento de los principales módulos del sistema.

Las pruebas realizadas demostraron una integración adecuada entre los distintos componentes de la aplicación y confirmaron que las funcionalidades implementadas responden correctamente a los requerimientos establecidos.

Asimismo, se logró mejorar la estabilidad general del sistema mediante la detección temprana de errores y la aplicación de correcciones durante el desarrollo.

10.6 Corrección de Errores Detectados

Durante el desarrollo se identificaron distintos errores relacionados con la autenticación, la comunicación entre módulos, consultas a la base de datos y validaciones de información.

Cada inconveniente fue analizado y corregido de forma progresiva a medida que avanzaba el proyecto, permitiendo mejorar la estabilidad del sistema.

En esta etapa, Claude fue una herramienta de gran ayuda para comprender errores complejos, interpretar mensajes de depuración y encontrar posibles soluciones. Su utilización facilitó el aprendizaje de nuevas tecnologías y

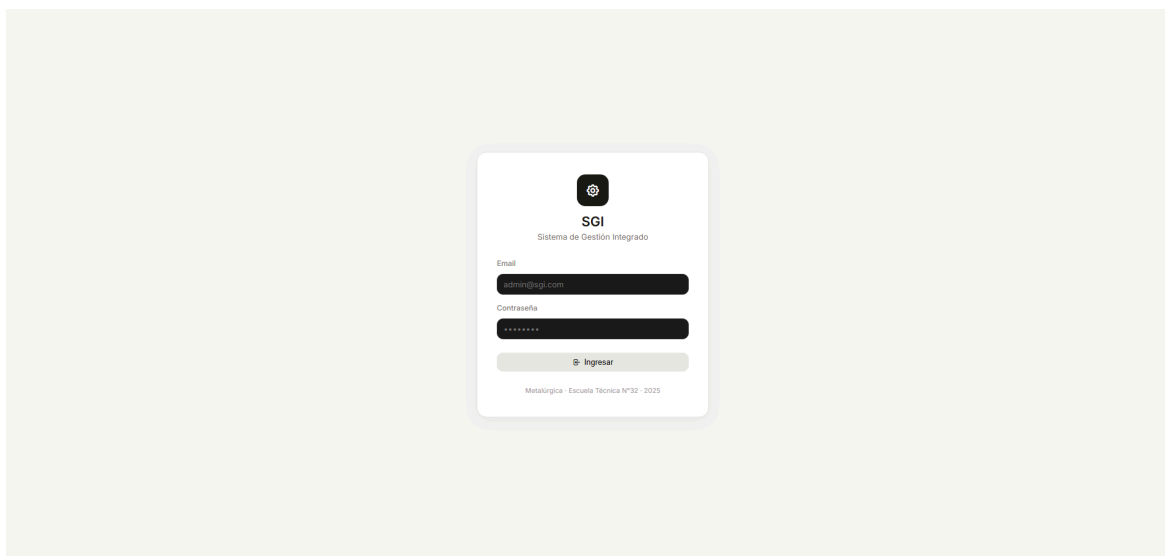
permitió resolver problemas técnicos que surgieron durante la implementación de diversas funcionalidades.

11. Manual de Usuario

11.1 Acceso al Sistema

Para acceder al sistema, el usuario debe ingresar sus credenciales en la pantalla de inicio de sesión. Una vez validadas por el sistema, se habilitará el acceso a los módulos correspondientes según el rol asignado.

Este mecanismo permite garantizar la seguridad de la información y controlar las funcionalidades disponibles para cada usuario.



11.2 Gestión de Usuarios

El módulo de usuarios permite administrar las cuentas registradas dentro de la aplicación y no hacerlo manualmente desde supabase o Postgresql.

Desde esta sección es posible crear nuevos usuarios, modificar información existente y gestionar los permisos asociados a cada rol. Estas funcionalidades facilitan el control de acceso y la organización de los usuarios dentro del sistema.

Panel Principal de Usuarios

SGI

Dashboard

Productos

Stock

Cuentas

Monitor

Reportes

Usuarios

Usuarios (0)

5 Usuarios cargados

5 Usuarios activos

1 Administradores

5 Roles en uso

Buscar por nombre, email o rol...

Todos los roles

Todos

USUARIO	EMAIL	ROL	ESTADO	ALTA		
1 Santino	Santino@gmail.com	Vendedor	Activo	25/6/2026		
2 Juan	Juan@hotmail.com	Vendedor	Activo	25/6/2026		
3 Consorti	consorti@bue.edu.ar	Administrador	Activo	25/6/2026		
4 Ramiro	ramirogarciaquechea@gmail.com	Comercio	Activo	25/6/2026		
5 Administrador	admin@sgi.com	Administrador	Activo	25/6/2026		

Modo oscuro

Administrador

Cerrar sesión

Card para crear Usuarios

Nuevo usuario

Cargá los datos iniciales para el acceso al sistema.

Nombre *

Rol

Nombre y apellido

Vendedor

Email *

Contraseña inicial *

usuario@sgi.com

Mínimo 6 caracteres

Estado

Inactivo

Cancelar

✓ Crear usuario

11.3 Gestión de Clientes

El módulo de clientes permite registrar y administrar la información de las personas o empresas que operan con la organización.

Los usuarios autorizados pueden consultar, agregar o editar datos comerciales necesarios para la gestión de pedidos y reportes.

Panel Principal de Clientes

RAZÓN SOCIAL	CUIT	TELÉFONO	EMAIL	LISTA DE PRECIO	DEUDA	
Construcciones Rivas S.R.L.	30-12345678-9	011-4444-5555	rivas@email.com	Mayorista	\$0,00	
Talleres Méndez S.A.	30-98765432-1	011-3333-4444	mendez@email.com	Mayorista	\$0,00	
Edesur	23456781239	1132433355	Edesur@gmail.com	Mayorista	\$0,00	

Card para crear un nuevo Cliente

Nuevo cliente

Razón social *

Empresa S.R.L.

CUIT

30-12345678-9

Lista de precio

Sin lista asignada

Teléfono

011-4444-5555

Email

cliente@email.com

Dirección

Av. Corrientes 1234, CABA

Cancelar

✓ Crear cliente

Card para editar a un Cliente

Editar cliente

Razón social *

Construcciones Rivas S.R.L.

CUIT

30-12345678-9

Lista de precio

Mayorista

Teléfono

011-4444-5555

Email

rivas@email.com

Dirección

Av. Corrientes 1234, CABA

Cancelar

✓ Guardar cambios

11.4 Gestión de Productos

A través del módulo de productos se administra el catálogo de artículos utilizados por la empresa.

Esta sección permite registrar nuevos productos, actualizar información existente y consultar los datos necesarios para la gestión de stock, producción y ventas.

Panel Principal de Clientes

SGI

Panel Principal

Dashboard

Pedidos

Productos

Stock

Clientes

Monitor

Reportes

Usuarios

Modo oscuro

Administrador

Cerrar sesión

Productos (4)

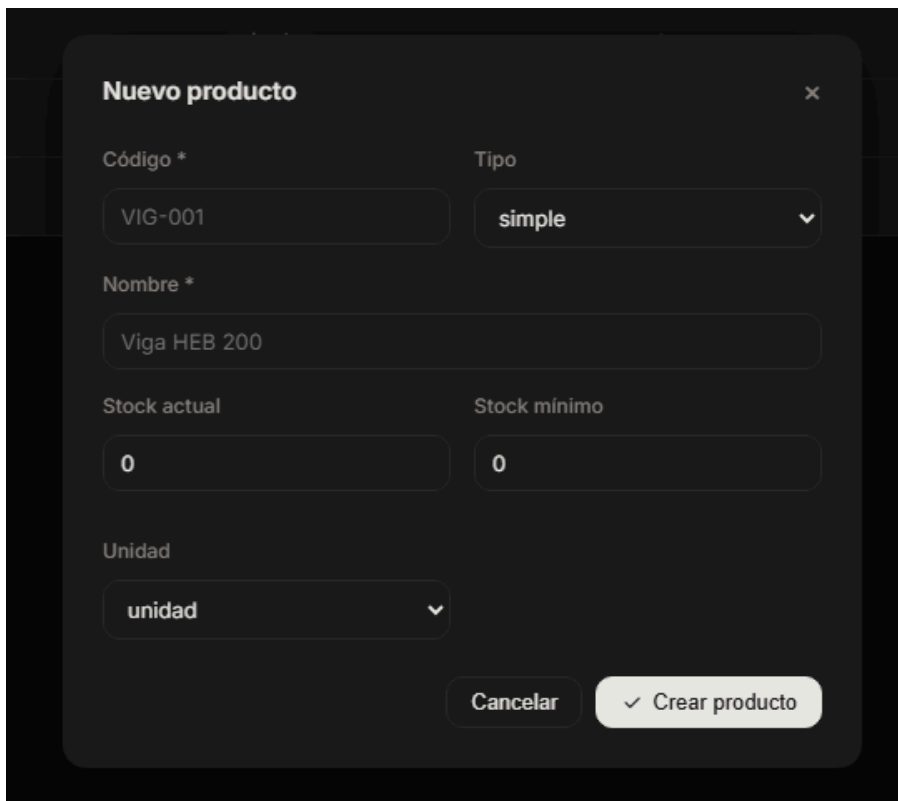
+ Nuevo producto

Buscar por nombre o código...

Todos Simple Compuesto

CODIGO	NOMBRE	TIPO	UNIDAD	STOCK ACTUAL	STOCK MINIMO	
ANG-001	Ángulo 50x50x5	simple	metro	100	20	
VIG-001	Viga HEB 200	simple	metro	1	10	
PER-001	Perno M16	simple	unidad	500	100	
KIT-001	Kit Estructura Galería	compuesto	kg	100	5	

Card para Crear un Nuevo Producto

Un formulario de creación de producto con un fondo oscuro. El título 'Nuevo producto' está en la parte superior izquierda con un botón de cerrar 'x' a la derecha. El formulario contiene los siguientes campos: 'Código *' con el valor 'VIG-001', 'Tipo' con un menú desplegable que muestra 'simple', 'Nombre *' con el valor 'Viga HEB 200', 'Stock actual' con el valor '0', 'Stock mínimo' con el valor '0', y 'Unidad' con un menú desplegable que muestra 'unidad'. En la parte inferior derecha hay dos botones: 'Cancelar' y '✓ Crear producto'.

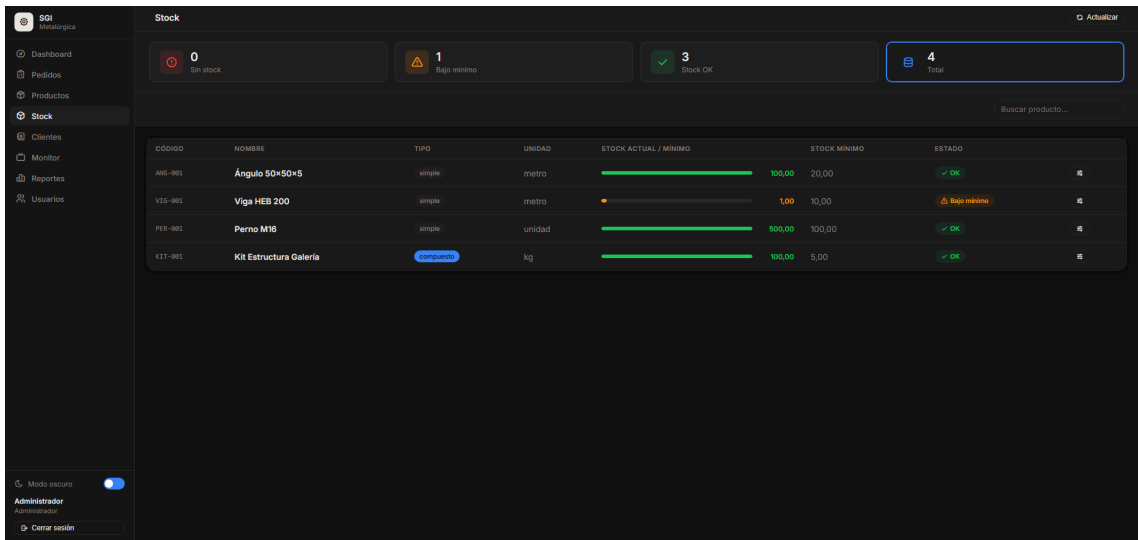
(También hay una card para editar pero para no poner tantas imagenes no la adjunte).

11.5 Gestión de Stock

El módulo de stock proporciona herramientas para controlar las existencias disponibles de cada producto..

Los usuarios pueden consultar cantidades disponibles, registrar movimientos de inventario y realizar el seguimiento de las variaciones producidas durante la operación diaria de la empresa. Esto sin tener que entrar a la base de datos para cambiarlo.

Panel Principal de Stock



Card para editar el Stock

Ajuste de stock

×

Ángulo 50×50×5

Stock actual100,00 metro

↑ Ingreso

↓ Egreso

Cantidad (metro)

0

Motivo (opcional)

Ej: Recepción de mercadería, ajuste inventario...

Cancelar

✓ Confirmar

11.6 Gestión de Pedidos

El módulo de pedidos permite registrar, consultar y administrar las órdenes realizadas por los clientes.

Cada pedido puede ser visualizado junto con su información asociada, facilitando el seguimiento de su estado y el control de las operaciones comerciales.

Panel Principal de Pedidos

SGI

Metaborga

Dashboard

Pedidos

Productos

Stock

Cuentas

Monitor

Reportes

Usuarios

Modo oscuro

Administrador

Administrar

Cerrar sesión

Pedidos de venta

+ Nuevo pedido

Lista

Nuevo pedido

Monitor

Todos (1)

Pendiente (2)

En Fabricación (1)

Control de Calidad (1)

Listo para Retirar (1)

Entregado (0)

NUMERO	CLIENTE	LISTA	TOTAL	ESTADO	SIGUIENTE ESTADO
PED-006	Construcciones Rivas S.R.L.	Mayorista	\$49.900,00	Pendiente	→ En Fabricación
PED-005	Construcciones Rivas S.R.L.	Minorista	\$50.000,00	Listo para Retirar	→ Entregado
PED-004	Edesur	Especial	\$990,00	Control de Calidad	→ Listo para Retirar
PED-003	Construcciones Rivas S.R.L.	Mayorista	\$0,00	En Fabricación	→ Control de Calidad
PED-001	Construcciones Rivas S.R.L.	Mayorista	\$30.000,00	Pendiente	→ En Fabricación

Panel para la creación de un nuevo Pedido

Pedidos de venta

+ Nuevo pedido

Lista

Nuevo pedido

Monitor

Cliente

Lista de precio

Fecha de entrega

Seleccioná un cliente...

Seleccioná una lista...

dd/mm/aaaa

PRODUCTOS

Seleccioná un producto...

1

+ Agregar

PRODUCTO	TIPO	CANT.	PRECIO UNIT.	SUBTOTAL
Añadí productos para comenzar.				

Total

\$0,00

Cancelar

✓ Confirmar pedido

Panel de el monitor

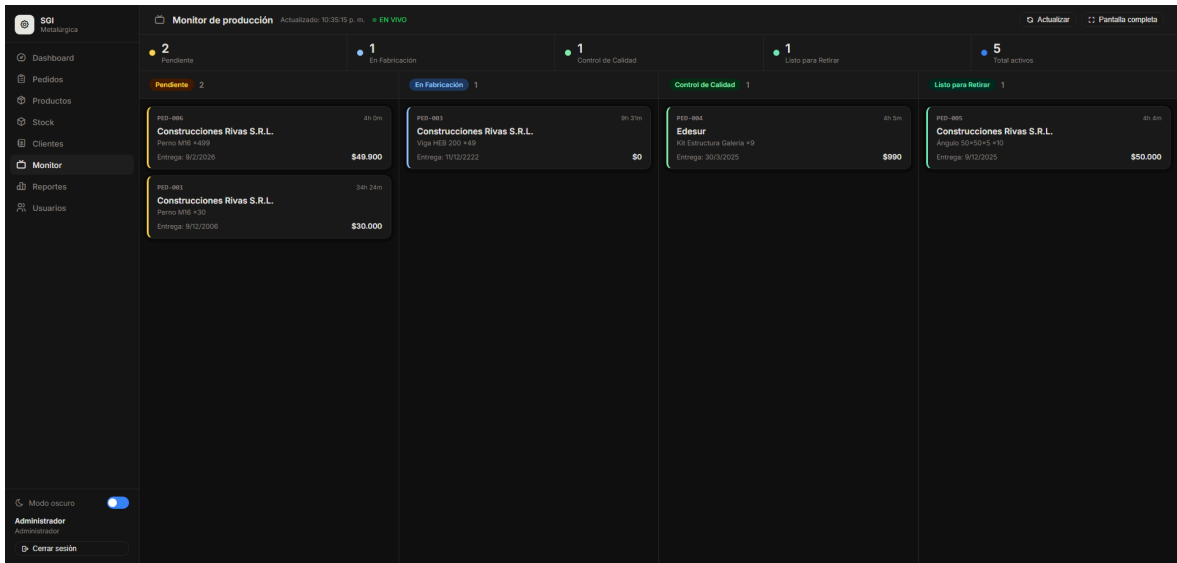


11.7 Seguimiento de Producción (Monitor)

La sección de producción permite monitorear el avance de los procesos productivos vinculados a los pedidos registrados.

Mediante esta funcionalidad los usuarios pueden consultar el estado actual de cada trabajo y realizar un seguimiento más eficiente de las actividades de fabricación.

Panel Principal de Monitor

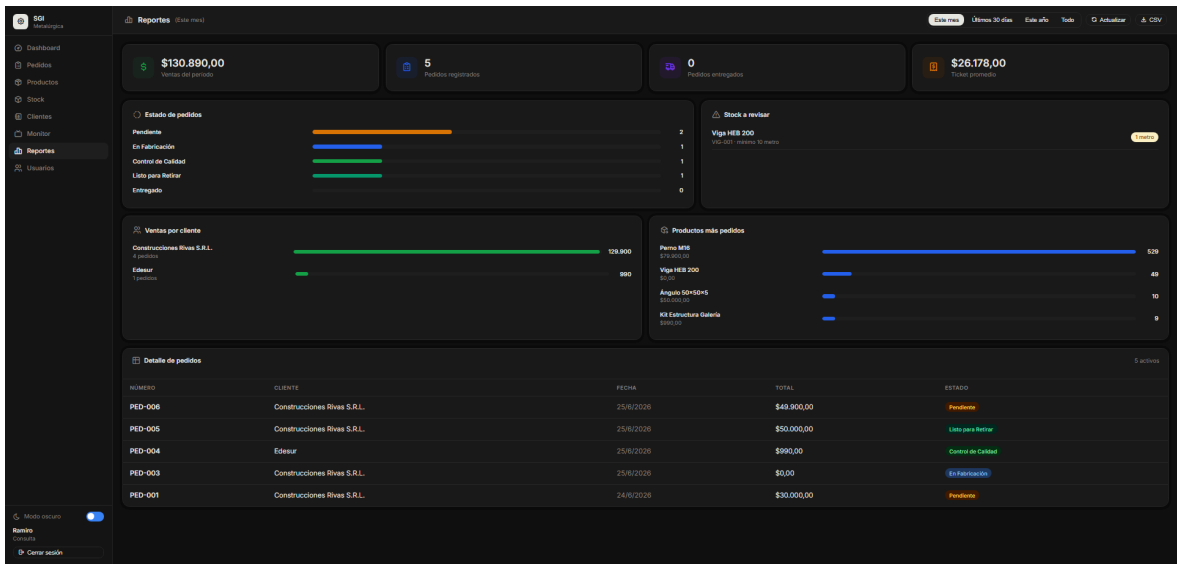


11.8 Generación de Reportes

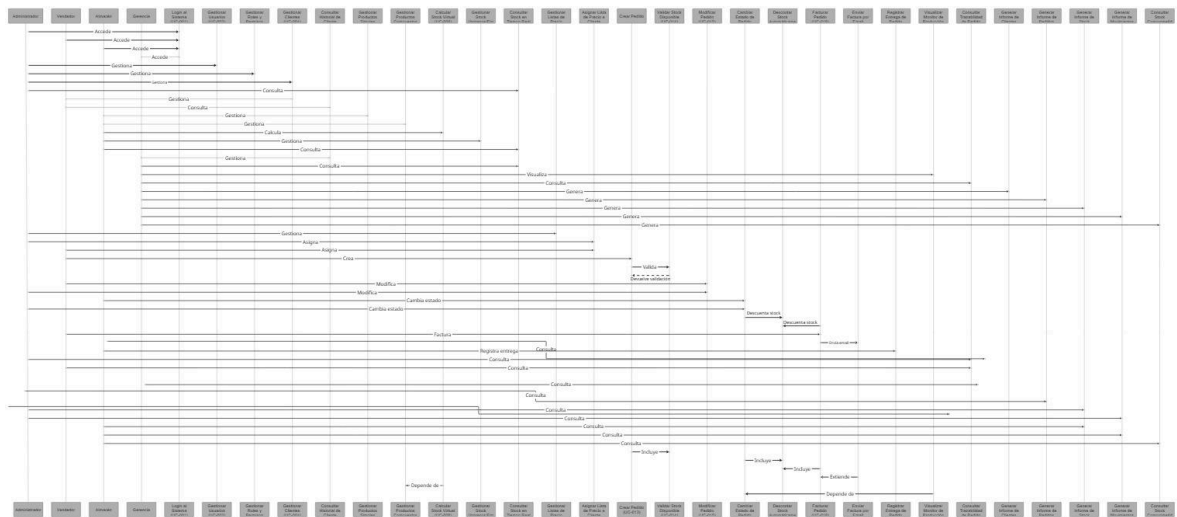
El módulo de reportes permite acceder a información consolidada del sistema mediante gráficos, tablas e indicadores.

Estos reportes facilitan el análisis de datos relacionados con clientes, productos, stock y pedidos, proporcionando información útil para la toma de decisiones y el control operativo de la organización.

Panel Principal de Reportes



11.9 Diagrama de caso de uso



12. Manual de Deploy

12.1 Requisitos Previos

Para ejecutar el Sistema de Gestión Integrado es necesario contar con los siguientes programas instalados:

- Visual Studio Code.

- Python.
- Node.js y npm.
- Acceso al proyecto completo (frontend y backend).

Actualmente la aplicación se encuentra configurada para ejecutarse en un entorno local (localhost), ya que al tratarse de un proyecto propio se desarrolló de esta manera para facilitar las pruebas y la demostración.

12.2 Configuración del Backend

- Para iniciar el servidor backend se debe ejecutar desde una terminal los siguientes comandos:

```
cd C:/Users/ramir/desktop/SGI/sgi-backend
```

```
python -m uvicorn main:app --reload
```

- Una vez iniciado el servidor, la API estará disponible en:

```
http://localhost:8000
```

- La documentación automática generada por FastAPI puede consultarse en:

```
http://localhost:8000/docs
```

- Esta interfaz permite visualizar y probar los distintos endpoints implementados en el sistema.
-

12.3 Configuración del Frontend

Para iniciar el frontend, se debe abrir una nueva terminal dentro de Visual Studio Code y ejecutar los siguientes comandos:

```
cd C:/Users/ramir/desktop/SGI/sgi-frontend
```

```
npm install
```

npm start

Una vez completado el proceso, la aplicación estará disponible desde el navegador en la dirección configurada por React.

El frontend se conectará automáticamente con el backend local para permitir el funcionamiento completo del sistema.

12.4 Configuración de la Base de Datos

La base de datos utilizada por el sistema fue desarrollada utilizando PostgreSQL mediante Supabase.

La conexión entre el backend y la base de datos se realiza mediante las credenciales configuradas en las variables de entorno del proyecto. **Esta conexión con las credenciales la tengo configurada yo, ya que se trata de un proyecto personal; sin embargo, si fuera un desarrollo real para una empresa, lo implementaría en PostgreSQL y con otro workspace que no fuera Supabase**, debido a que esta opción resulta bastante simple para un proyecto pequeño como este.

Gracias a esta configuración, toda la información generada por la aplicación se almacena de forma centralizada y segura.

12.5 Variables de Entorno

El sistema utiliza variables de entorno para almacenar configuraciones sensibles necesarias para su funcionamiento.

Entre ellas se incluyen datos relacionados con:

- **Conexión a la base de datos.**
- **Credenciales de Supabase.**
- **Configuración de autenticación.**
- **Parámetros de seguridad.**

Este mecanismo evita exponer información sensible directamente dentro del código fuente.

12.6 Publicación en Producción

Actualmente el proyecto se encuentra configurado para ejecutarse en un entorno local (localhost), siendo utilizado principalmente para desarrollo, pruebas y demostraciones.

La arquitectura implementada permite que en futuras versiones pueda desplegarse en un servidor de producción, facilitando el acceso remoto de múltiples usuarios y ampliando las capacidades del sistema. Actualmente, el sistema se encuentra en fase BETA.

Por el momento, la ejecución local mediante Visual Studio Code resulta suficiente para demostrar el correcto funcionamiento de todas las funcionalidades desarrolladas durante el proyecto.

13. Resultados del Proyecto

13.1 Funcionalidades Implementadas

Durante el desarrollo del Sistema de Gestión Integrado se logró implementar la mayoría de las funcionalidades previstas en la etapa de análisis y planificación, y todo ello en un lapso muy corto de tiempo, lo que representa un resultado especialmente significativo considerando la amplitud del alcance del proyecto.

Entre las principales funcionalidades desarrolladas se encuentran:

- Sistema de autenticación mediante JWT.
- Gestión de usuarios y roles.
- Gestión de clientes.
- Gestión de productos simples y compuestos.
- Control y seguimiento de stock.
- Gestión de pedidos.
- Seguimiento de producción.
- Generación de reportes.
- Dashboard principal con información centralizada.
- Integración entre frontend, backend y base de datos.

13.2 Comparación con los Requerimientos Iniciales

Al comparar los resultados obtenidos con los requerimientos definidos al inicio del proyecto, se observa un alto grado de cumplimiento de los objetivos planteados, especialmente si se considera que el sistema fue desarrollado en un lapso muy corto de tiempo.

Las funcionalidades principales previstas durante el análisis fueron desarrolladas e integradas correctamente dentro del sistema. Además, durante el proceso se incorporaron mejoras y ajustes que permitieron optimizar la experiencia de usuario y fortalecer la estructura general de la aplicación, todo ello dentro de un tiempo de ejecución bastante acotado.

Si bien algunos aspectos pueden continuar evolucionando en futuras versiones, el proyecto logró satisfacer los requerimientos esenciales establecidos en las primeras etapas de planificación, lo cual resulta aún más destacable por la rapidez con la que fue llevado a cabo.

13.3 Beneficios Obtenidos

La implementación del Sistema de Gestión Integrado permitió centralizar información que anteriormente se encontraba distribuida en distintos procesos y registros.

Entre los principales beneficios obtenidos se destacan:

- Mayor organización de la información.
- Mejor control de usuarios y permisos.
- Gestión más eficiente de clientes y productos.
- Seguimiento actualizado del stock.
- Control de pedidos y producción.
- Acceso rápido a reportes e indicadores.
- Reducción de errores operativos.
- Base sólida para futuras ampliaciones del sistema.

Además los beneficios técnicos obtenidos, el proyecto representó una importante experiencia de aprendizaje personal, permitiéndome adquirir

conocimientos en React, FastAPI, PostgreSQL, Supabase, autenticación JWT y desarrollo de aplicaciones web completas utilizando arquitecturas modernas. Asimismo, el hecho de haberlo desarrollado en un lapso muy corto de tiempo. Además se demuestra capacidad de organización, adaptación y resolución eficiente de problemas durante todo el proceso.

14. Conclusiones

14.1 Objetivos Alcanzados

El principal objetivo del proyecto consistía en desarrollar un Sistema de Gestión Integrado capaz de centralizar distintos procesos operativos dentro de una única aplicación. A lo largo del desarrollo se logró implementar las funcionalidades fundamentales previstas durante la etapa de análisis, permitiendo gestionar usuarios, clientes, productos, stock, pedidos y producción de forma integrada.

Cabe destacar que este proyecto fue desarrollado en un lapso de tiempo muy corto, considerando además que cuento con un trabajo aparte, por lo que el proceso representó un desafío importante en cuanto a organización y dedicación. Aun así, me siento muy feliz y contento de haber podido llevarlo a cabo, ya que se trata de mi primer proyecto web de gran magnitud.

Los resultados obtenidos demuestran que el sistema cumple con los objetivos planteados inicialmente y proporciona una base sólida para futuras mejoras y ampliaciones.

14.2 Aprendizajes Adquiridos

Este proyecto representó una experiencia de aprendizaje muy importante tanto a nivel técnico como personal. Además, tuve que desarrollarlo en un lapso corto de tiempo, y aprender todo desde cero fue un desafío muy difícil.

Durante su desarrollo adquirí conocimientos relacionados con React, FastAPI, PostgreSQL, Supabase, autenticación mediante JWT, consumo de APIs REST y desarrollo de aplicaciones web bajo una arquitectura cliente-servidor, aunque

ya había tenido experiencia haciendo proyectos con Java, armarlo fue totalmente distinto.

Además, aprendí a investigar tecnologías nuevas, resolver problemas de forma autónoma, interpretar documentación técnica y aplicar buenas prácticas de programación en un proyecto de gran tamaño.

14.3 Dificultades Encontradas

Uno de los principales desafíos fue trabajar con tecnologías que inicialmente no formaban parte de mis conocimientos previos.

La implementación de la autenticación, el manejo de permisos y la integración con la base de datos requirieron un importante proceso de aprendizaje y adaptación. Lo que más me costó fue la estructuración del código tanto en frontend como en backend, ya que al principio no sabía cómo organizar correctamente cada parte del proyecto. También tuve dificultades para realizar la conexión entre frontend y backend, así como para comprender la lógica cliente-servidor, porque era un tema nuevo para mí. A esto se sumó el manejo de FastAPI, que implicó aprender su estructura, sus rutas y la forma correcta de comunicarlo con el resto del sistema.

Durante esta etapa, herramientas como Claude fueron de gran ayuda para comprender conceptos complejos, resolver errores y acelerar el aprendizaje necesario para avanzar en el desarrollo del sistema.

14.4 Mejoras Futuras

Si bien el sistema cumple con los objetivos principales planteados para esta versión, existen diversas mejoras que podrían incorporarse en futuras actualizaciones.

Entre ellas se destacan:

- Despliegue en un entorno de producción.
- Optimización de rendimiento.
- Incorporación de nuevos reportes e indicadores.

- Implementación de notificaciones automáticas.
- Mejora de la experiencia de usuario.
- Desarrollo de nuevas funcionalidades según las necesidades de la organización.

15. Bibliografía

Durante el desarrollo del Sistema de Gestión Integrado se consultaron distintas fuentes y herramientas de apoyo para implementar las funcionalidades del proyecto.

15.1 Documentación Oficial

- React documentacion
- Fast Api documentación
- SQL/Postgre documentacion.
- Supabase
- JWT documentacion
- Python documentacion
- [Node.js](#) documentacion

15.2 Recursos de Aprendizaje

- Midudev y videos de youtube
- W3schools que tiene todas las documentaciones.
- Datacamp que tambien tiene todas las documentaciones.

15.3 Herramientas Utilizadas

- Visual Studio Code

- GitHub
 - Supabase
 - Claude IA
 - ChatGPT (OpenAI)
-

15.4 Referencias Académicas

Las decisiones de diseño e implementación realizadas durante el proyecto se basaron en principios de desarrollo de software, arquitectura cliente-servidor, bases de datos relacionales y metodologías ágiles estudiadas durante la formación técnica y complementadas mediante investigación autónoma.

La combinación de documentación oficial, recursos educativos y herramientas de asistencia permitió desarrollar una solución funcional, escalable y alineada con las buenas prácticas actuales del desarrollo beta del software.

16. Código

16.1 Capturas del Sistema Del Frontend

Pantalla de Inicio de Sesión

```
43      /* Formulario */
44      <form onSubmit={handleSubmit}>
45        <div className="form-group">
46          <label>Email</label>
47          <input
48            type="email"
49            placeholder="admin@sgi.com"
50            value={form.email}
51            onChange={e => setForm({ ...form, email: e.target.value })}
52            required
53            autoFocus
54          />
55        </div>
56        <div className="form-group">
57          <label>Contraseña</label>
58          <input
59            type="password"
60            placeholder="*****"
61            value={form.password}
62            onChange={e => setForm({ ...form, password: e.target.value })}
63            required
64          />
65        </div>
66        <button
67          type="submit"
68          className="btn btn-primary"
69          style={{ width: "100%", justifyContent: "center", marginTop: 8 }}
70          disabled={cargando}
71        >
72          {cargando
73            ? <i className="ti ti-loader-2" style={{ animation: "spin .6s linear infinite" }} /> Ingresando...</>
74            : <i className="ti ti-login" /> Ingresar</>
75          }
76        </button>
77      </form>
78
79      <p style={styles.hint}>
80        Metalúrgica · Escuela Técnica N°32 · 2025
81      </p>
82    </div>
83  </div>
84  );
85 }
```

Es la parte de formulario, donde tienen que iniciar sesión.

Gestión de Pedidos

```
// Cargar pedidos
const cargarPedidos = useCallback(async () => {
  setCargando(true); setError(null);
  try { setPedidos(await api("/pedidos")); }
  catch(e){ setError(e.message); }
  finally { setCargando(false); }
}, [api]);

// Cargar datos para el formulario
const cargarFormData = useCallback(async () => {
  try {
    const [c, l, p] = await Promise.all([
      api("/clientes").catch(()=>[]),
      api("/listas-precio").catch(()=>[]),
      api("/productos").catch(()=>[]),
    ]);
    setClientes(c); setListas(l); setProductos(p);
  } catch {}
}, [api]);

useEffect(() => { cargarPedidos(); cargarFormData(); }, [cargarPedidos, cargarFormData]);

// Filtrar lista
const lista = filtro==="todos" ? pedidos : pedidos.filter(p=>p.estado===filtro);

// Cambiar estado
const cambiarEstado = async (id, estado) => {
  console.log("Enviando estado:", JSON.stringify(estado)); // ← VE QUÉ ENVÍA
  try {
    const actualizado = await api(`/pedidos/${id}/estado`, {
      method:"PATCH",
      body:JSON.stringify({
        estado: estado.trim(), // ← Elimina espacios
        observacion: ""
      })
    });
    setPedidos(prev => prev.map(p => p.id===id ? actualizado : p));
    if (detalle?.id===id) setDetalle(actualizado);
  } catch(e){
    alert("Error: " + e.message);
  }
}
```

Esta es la parte de carga de pedidos, que es una herramienta esencial en la parte de pedidos.

Reportes: conexión entre back y front.

```
src > pages > Reportes.jsx > ...
1  import { useCallback, useEffect, useMemo, useState } from "react";
2
3  const API = process.env.REACT_APP_API_URL || "http://localhost:8000/api";
4
5  const ESTADOS = [
6    "Pendiente",
7    "En Fabricación",
8    "Control de Calidad",
9    "Listo para Retirar",
10   "Entregado",
11 ];
12
13 const BADGE = {
14   "Pendiente": "badge-pendiente",
15   "En Fabricación": "badge-fabricacion",
16   "Control de Calidad": "badge-calidad",
17   "Listo para Retirar": "badge-listo",
18   "Entregado": "badge-entregado",
19 };
20
21 const PERIODOS = {
22   mes: "Este mes",
23   treinta: "Últimos 30 días",
24   año: "Este año",
25   todos: "Todo",
26 };
27
28 function useApi() {
29   const token = localStorage.getItem("sgi_token");
30   return useCallback(async (endpoint, options = {}) => {
31     const res = await fetch(`${API}${endpoint}`, {
32       ...options,
33       headers: {
34         "Content-Type": "application/json",
35         ...(token ? { Authorization: `Bearer ${token}` } : {}),
36         ...options.headers,
37       },
38     });
39     if (!res.ok) {
40       const e = await res.json().catch(() => ({}));
41       throw new Error(e.detail || `Error ${res.status}`);
42     }
43     return res.status === 204 ? null : res.json();
44   }, [token]);
45 }
46
47 function money(n) {
48   return "$" + Number(n || 0).toLocaleString("es-AR", {
```

Esta parte del código no es esencial pero muestra la conexión entre FastApi y el backend.

16.2 Capturas del Sistema Del Backend

Usuarios: Lista de usuarios, lista de usuarios, crear usuario y obtener usuario.

```
@router.get("/")
def listar_usuarios(db: Session = Depends(get_db)):
    usuarios = db.query(Usuario).order_by(Usuario.id.desc()).all()
    return [usuario_out(usuario) for usuario in usuarios]

@router.get("/roles")
def listar_roles(db: Session = Depends(get_db)):
    roles = db.query(Rol).order_by(Rol.nombre.asc()).all()
    return roles

@router.get("/{id}")
def obtener_usuario(id: int, db: Session = Depends(get_db)):
    usuario = db.query(Usuario).filter(Usuario.id == id).first()
    if not usuario:
        raise HTTPException(status_code=404, detail="Usuario no encontrado")
    return usuario_out(usuario)

@router.post("/")
def crear_usuario(datos: UsuarioCreate, db: Session = Depends(get_db)):
    existe = db.query(Usuario).filter(Usuario.email == datos.email).first()
    if existe:
        raise HTTPException(status_code=400, detail="Ya existe un usuario con ese email")

    rol = obtener_o_crear_rol(datos.rol, db)
    usuario = Usuario(
        nombre=datos.nombre,
        email=datos.email,
        password_hash=pwd_context.hash(datos.password),
        rol_id=rol.id,
        activo=datos.activo,
    )
    db.add(usuario)
    db.commit()
    db.refresh(usuario)
    return usuario_out(usuario)
```

Productos: Crear Producto

```
class ProductoCreate(BaseModel):
    codigo: str
    nombre: str
    tipo: str = "simple"
    stock_actual: Decimal = 0
    stock_minimo: Decimal = 0
    unidad: str = "unidad"

@router.get("/")
def listar_productos(db: Session = Depends(get_db)):
    return db.query(Producto).all()

@router.get("/{id}")
def obtener_producto(id: int, db: Session = Depends(get_db)):
    return db.query(Producto).filter(Producto.id == id).first()

@router.get("/{id}/stock-virtual")
def stock_virtual(id: int, db: Session = Depends(get_db)):
    from app.services.stock_service import calcular_stock_virtual
    return { "producto_id": id, "stock_disponible": calcular_stock_virtual(id, db) }

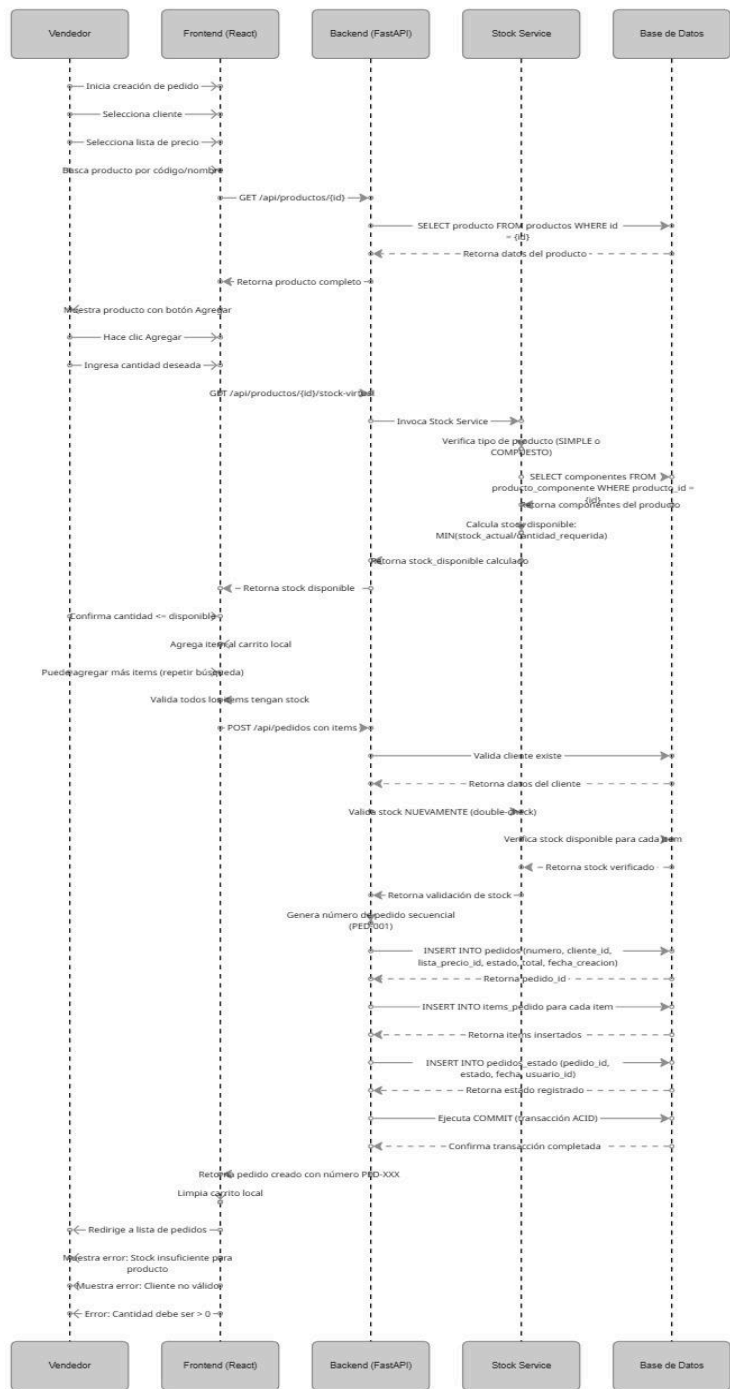
@router.post("/")
def crear_producto(datos: ProductoCreate, db: Session = Depends(get_db)):
    producto = Producto(**datos.model_dump())
    db.add(producto)
    db.commit()
    db.refresh(producto)
    return producto

from fastapi import HTTPException
from pydantic import BaseModel
```

Cientes: Crear cliente

```
1 from fastapi import APIRouter, Depends
2 from sqlalchemy.orm import Session
3 from pydantic import BaseModel
4 from typing import Optional
5 from app.database import get_db
6 from app.models.cliente import Cliente
7
8 router = APIRouter()
9
10 class ClienteCreate(BaseModel):
11     razon_social: str
12     cuit: Optional[str] = None
13     telefono: Optional[str] = None
14     email: Optional[str] = None
15     direccion: Optional[str] = None
16     lista_precio_id: Optional[int] = None
17
18 @router.get("/")
19 def listar_clientes(db: Session = Depends(get_db)):
20     return db.query(Cliente).all()
21
22 @router.get("/{id}")
23 def obtener_cliente(id: int, db: Session = Depends(get_db)):
24     return db.query(Cliente).filter(Cliente.id == id).first()
25
26 @router.post("/")
27 def crear_cliente(datos: ClienteCreate, db: Session = Depends(get_db)):
28     cliente = Cliente(**datos.model_dump())
29     db.add(cliente)
30     db.commit()
31     db.refresh(cliente)
32     return cliente
33
34 @router.put("/{id}")
35 def editar_cliente(id: int, datos: ClienteCreate, db: Session = Depends(get_db)):
36     cliente = db.query(Cliente).filter(Cliente.id == id).first()
37     if not cliente:
38         from fastapi import HTTPException
39         raise HTTPException(status_code=404, detail="Cliente no encontrado")
40     for k, v in datos.model_dump().items():
41         setattr(cliente, k, v)
42     db.commit()
43     db.refresh(cliente)
44     return cliente
```

16.2 Diagrama de Secuencia



FIN
